

Relatório de Auditoria de Segurança

FazendaApp

Data: 02/09/2026

Escopo: Backend FastAPI/SQLModel completo (backend/fazenda/api/routers/**, ~95 arquivos de rota, auth.py, regras de negócio em fazenda/rules/**), frontend Next.js/React/Capacitor completo (frontend/app, frontend/components, frontend/lib), configuração de deploy (Railway, Vercel, GitHub Actions) e histórico git completo.

Nota metodológica — mapeamento das categorias para esta stack

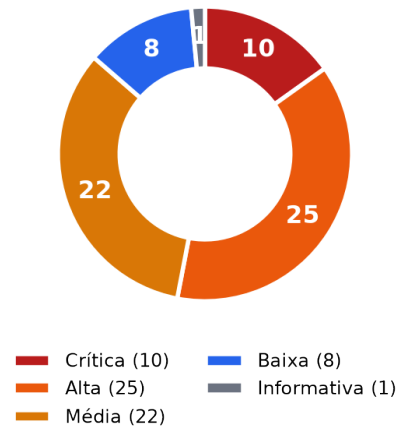
- Detecção de stack: FastAPI + SQLModel/SQLAlchemy (Postgres em produção via Railway, SQLite em testes); autenticação própria via token HMAC assinado (fazenda/auth.py — não é JWT de biblioteca); frontend Next.js 16 + React 19 com wrapper Capacitor para Android; deploy via Railway (backend+Postgres) e Vercel (frontend); CI via GitHub Actions. Não há Supabase RLS — Supabase é usado só como armazenamento de arquivos, com a service_role key mantida só no backend.
- Categoria 1 (BANCO SEM TRANCA) mapeada para: isolamento por fazenda_id em vez de RLS. Mecanismo de referência identificado em fazenda/auth.py — get_fazenda_atual_id (leitura, tolera None para instalações legadas/single-tenant) e get_fazenda_id_escrita (escrita, recusa com 409 em vez de gravar fazenda_id nulo). O padrão correto de leitura é 'if fazenda_id is not None: query.where(...)'; o padrão incorreto recorrente é escrita usando a dependência de leitura, ou checagem de posse que tolera fazenda_id=None indevidamente.
- Categoria 2 (PERMISSÃO NO NAVEGADOR) mapeada para: todo gate de papel do frontend (lib/api.ts: ehDono/ehAdmin/podePublicarMaterias/etc.) cruzado individualmente com o endpoint backend correspondente, rota por rota.
- Categoria 3 (IDOR) mapeada para: toda rota que busca/altera/exclui um objeto por id (path/query/body) — os ~640 handlers de rota foram percorridos integralmente, divididos em 7 lotes por arquivo/linha, sem amostragem.
- Categoria 4 (CHAVES EXPOSTAS) varrida em todo o código-fonte, configs, CI, docs, e no histórico git completo (1616 commits) via buscas delimitadas (git log -S/--diff-filter, sem log -p --all irrestrito).
- Categoria 5 (XSS) mapeada para: dangerouslySetInnerHTML/v-html equivalentes e URLs controladas por usuário no frontend; interpolação de input de usuário em HTML de e-mail/template/PDF sem escape no backend.
- Todo achado foi verificado por leitura direta do código-fonte (não há achado especulativo). Os dois achados mais críticos (auth.py e painel_cowdata_usuarios.py) foram re-verificados manualmente e corrigidos nesta mesma sessão, com testes de regressão.

Resumo executivo

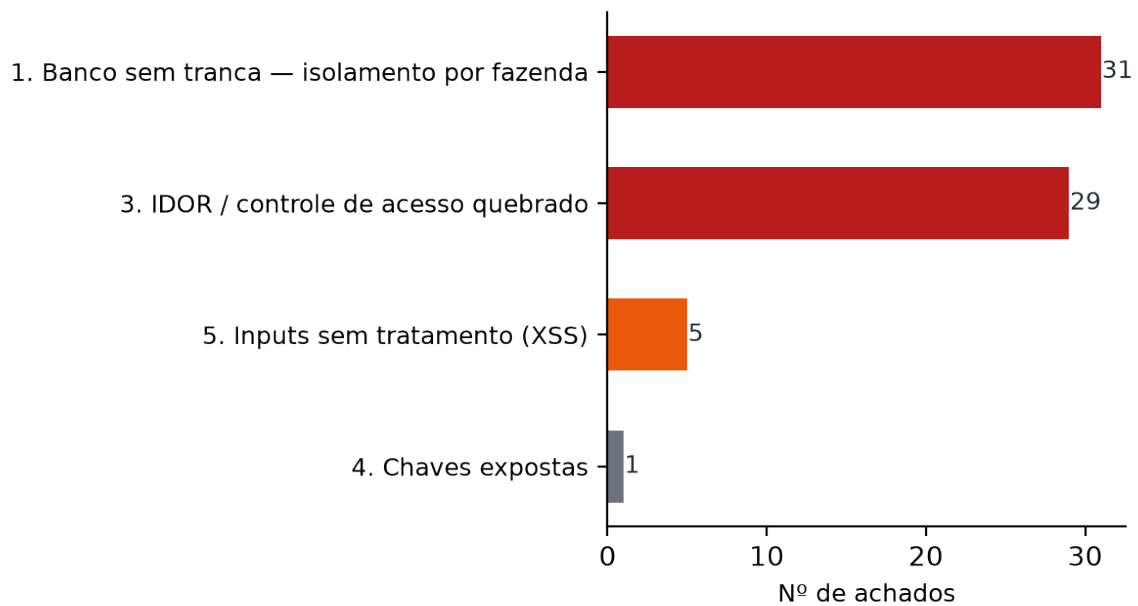
Foram revisados sistematicamente ~640 handlers de rota, em ~95 arquivos de rota backend (auth.py, main.py e regras de negócio inclusos) arquivos de backend, mais o frontend e a configuração de segredos/deploy. Este relatório registra 66 achado(s) acionável(is) e 22 pontos fortes verificados (evidência de cobertura da auditoria).

Achados por severidade

Severidade	Qtd.
Crítica	10
Alta	25
Média	22
Baixa	8
Informativa	1
Total	66



Achados por categoria



Pontos fortes

O que foi verificado no código e está corretamente protegido — evidência direta de cobertura da auditoria (cada item cita onde a proteção foi conferida).

Dependência `get_fazenda_id_escrita` recusa gravar `fazenda_id` nulo

`backend/fazenda/auth.py:248-317`

Helper bem desenhado e documentado: em vez de silenciosamente gravar `fazenda_id=NULL` para um token legado/ambíguo, recusa com 409. É o padrão correto que dezenas de rotas de criação já usam corretamente (`financeiro.py:criar_lancamento`, `rh_contratos.py:criar_contrato/criar_diaria/criar_vale_avulso`, `recria.py` inteiro, `formulacao_dietas.py`, `estoque.py`, etc.) — as ocorrências onde NÃO é usado (catalogadas nesta auditoria) são a exceção, não a regra.

`AUTH_SECRET` tem guard de produção real e testado, chamado no boot

`backend/fazenda/auth.py:32-53`; `backend/main.py:187`

`exigir_segredo_de_producao()` roda como a primeira linha do lifespan da FastAPI, antes de criar tabelas ou rodar seed — o app se recusa a subir em produção (Postgres) com o segredo de desenvolvimento padrão.

Comparações de segredo em tempo constante em todo o mecanismo de auth

`backend/fazenda/auth.py:107,184`

`hmac.compare_digest` usado tanto na assinatura HMAC do token quanto na comparação de hash PBKDF2 de senha — nenhuma comparação de `string ==` em segredo em todo o arquivo.

Sem senha de admin hardcoded no seed

`backend/fazenda/auth.py:734-763` (`seed_admin`)

Se `ADMIN_PASS` não está definida, gera um token aleatório (`secrets.token_urlsafe`) e imprime uma única vez no log — nunca uma senha fixa de fallback.

Todas as 86 rotas do Pannel CowData corretamente gated

`painel_cowdata.py`, `painel_cowdata_cadastros.py`, `painel_cowdata_farmacia.py`, `painel_cowdata_parametros.py`, `painel_cowdata_usuarios.py`, `cofre_acesso.py`

Verificado rota por rota: todas usam `exigir_area_painel_cowdata(area)` ou `exigir_dono` — nenhuma rota é alcançável por um operador comum de fazenda.

Webhooks de pagamento Asaas e ZapSign comparam segredo e reconfirmam status

`backend/fazenda/api/routers/asaas.py:143-185`; `zapsign.py:25-28`

`asaas_webhook` compara o token contra um segredo do servidor e nunca confia no campo 'status' do payload sem reconfirmar via chamada autenticada à API do Asaas antes de mudar qualquer estado — contraste direto com o gap encontrado em `cobranca.py` (webhook do Banco do Brasil).

Nenhum `dangerouslySetInnerHTML/v-html` com dado de usuário no frontend

`frontend/app/layout.tsx:57-63` (único uso real, interpola só constante estática de cor)

Todo o restante das ocorrências de `dangerouslySetInnerHTML` encontradas por grep estão em `node_modules/.next` (build artifacts), não em código da aplicação. Não há biblioteca de renderização de markdown no frontend — sem essa superfície de XSS.

Todo gate de permissão do frontend tem enforcement backend independente e verificado

frontend/lib/api.ts (ehDono/ehAdmin/podePublicarMaterias/etc.) cruzado com financeiro.py, parametros.py, producao.py, cadastro/animais.py, exclusoes.py, aprovacoes.py, auth.py, news.py, cofre_acesso.py

Nenhum caso de categoria 2 (permissão definida só no navegador) foi confirmado nesta auditoria — todo controle privilegiado visto no frontend (Patrimônio, Parâmetros, renumerar animal, exclusão administrativa, Aprovações, Relatório de acessos, publicar matéria, Auditoria CowData) tem a dependência backend equivalente. Os comentários em lib/api.ts citam explicitamente o arquivo/função backend correspondente, evidência de alinhamento deliberado.

Exclusão de animal/objeto sempre passa por fluxo de aprovação para não-admin

backend/fazenda/api/routers/exclusoes.py:1271-1374

POST /exclusoes/confirmar checa o papel do usuário no próprio servidor (não confia em nenhuma flag do cliente) antes de decidir entre excluir de fato ou só criar uma SolicitacaoExclusao pendente; aprovar/rejeitar são exigir_admin.

Auditoria de atividade re-valida posse do usuário consultado, com 404 (não 403)

backend/fazenda/api/routers/auditoria.py:132-141

listar_atividades revalida que o usuario_id da query pertence à fazenda do chamador antes de retornar qualquer atividade, retornando 404 (não 403) para evitar confirmar a existência do id — comentário no código explica exatamente essa escolha.

Bloqueio de escrita RH durante sessão de suporte é mantido e documentado

backend/main.py:402-458, 489-567 (_bloquear_modos_suporte, _PREFIXOS_RH_MODALIDADE_SUORTE)

Bloqueia POST/PUT/DELETE em todos os prefixos de RH durante uma sessão CowData, e leitura abaixo do sigilo 'total' — com histórico documentado de um bug anterior corrigido (RH escrevível em modo suporte) e toda tentativa bloqueada é auditada.

Bounds de escopo do contratante re-validados no servidor

backend/fazenda/api/routers/fazendas.py:97-105 (_validar_escopo_contratante)

Toda mutação escopada a contratante confere que o fazenda_id da URL bate com a fazenda selecionada no token do próprio contratante, fechando uma lacuna que a checagem de papel sozinha deixaria aberta.

Toda a superfície de plano/contrato de fazenda é exigir_dono

backend/fazenda/api/routers/fazendas.py (definir_contrato, aprovar_contrato, suspender_contrato, atualizar_preco_modulo, criar_fazenda, editar_fazenda)

Uma fazenda-cliente não pode se auto-conceder módulos/plano que não contratou, nem ler/alterar o contrato de outro tenant.

Escopo de suporte (fazenda/nível de sigilo) é gravado no token assinado, nunca em parâmetro do cliente

backend/fazenda/api/routers/cofre_acesso.py:135-153; fazenda/auth.py:123-177,234-245

fazenda_id/nivel_sigilo/sessao_suporte_id são gravados no payload HMAC-assinado no momento de abrir a sessão de suporte; get_fazenda_atual_id só lê esse payload verificado, nunca um parâmetro de requisição — um token de suporte não pode ser redirecionado para outra fazenda por manipulação de parâmetro.

Correção documentada e testada de vazamento de estoque de sêmen (o modelo a seguir)

backend/fazenda/api/routers/relatorio_acasalamento.py:25-30

Corrige explicitamente a mesma classe de vazamento encontrada nesta auditoria em relatorios.py/nao_conformidades.py, citando o teste de regressão nomeado (tests/test_isolamento_relatorios_fornecedor.py) — prova de que a equipe já caça e corrige ativamente esta classe de bug.

Secagem já corrigida contra o mesmo padrão de tolerância a fazenda_id nulo

backend/fazenda/api/routers/reproducao.py:1281-1305,1315-1329

atualizar_secagem usa checagem estrita (!=) em vez de tolerar fazenda_id=None, com docstring citando o bug corrigido ('G5') — o mesmo fix não foi replicado em Servico/Parto no mesmo arquivo (ver achados).

Farmácia: correção documentada do vazamento global→fazenda em atualizar_marca/atualizar_indicacao

backend/fazenda/api/routers/farmacia.py:218-263

O código e seu comentário nomeiam exatamente a vulnerabilidade (token ambíguo editando direto o catálogo global) e trocam para get_fazenda_id_escrita — só não foi replicado nos DELETES irmãos (ver achados).

Segredo Supabase (service_role) nunca chega ao frontend

backend/fazenda/config.py:62; frontend/lib/api.ts:2

supabase_service_key só é lido em fazenda/rules/supabase_storage.py, no backend. O único NEXT_PUBLIC_* de todo o frontend é NEXT_PUBLIC_API_URL, a URL pública do backend — não um segredo.

Nenhum segredo real encontrado em 1616 commits de histórico git

histórico completo do repositório

Buscas delimitadas (adição de arquivo com nome de segredo, -S em padrões de atribuição de AUTH_SECRET/senha/api_key, -S'-----BEGIN') não encontraram nenhum segredo real jamais commitado, atual ou apagado.

.gitignore cobre corretamente arquivos de ambiente

backend/.env, frontend/.gitignore (.env*)

Única exceção é android/app/debug.keystore, chave de assinatura de debug Android com senha pública conhecida ('android'), nunca usada para build de release — exceção justificada.

Idempotência bancária (BB) usa PIX/nossoNumero, mas o pagamento em si segue reconfirmação

backend/fazenda/api/routers/cartao_credito.py:360-364; rh_vale_item.py:109-174

Toda referência cruzada (Pessoa, Empreitada/Contrato/Diária) usada para montar um vale é re-checada contra fazenda_id antes do uso, incluindo a checagem 'a origem escolhida não é desta pessoa'.

Renumeração de animal é admin-gated e seguramente global por design

backend/fazenda/api/routers/cadastro/animais.py:310-403

Depends(exigir_admin); a falta de filtro por fazenda_id na cascata é deliberada e segura, pois Animal.numero é globalmente único — o animal já foi checado por posse antes.

Pontos fracos — riscos centrais

- Escalação de privilégio para dono da plataforma via e-mail dono-equivalente em duas rotas (auth.py e painel_cowdata_usuarios.py) — JÁ CORRIGIDO nesta sessão.

- Confirmação/desconfirmação de protocolos na Agenda (IATF, indução, sanitário, customizado, lida, BST) aceita qualquer id sem checar a fazenda do chamador — grava/apaga tratamento sanitário de qualquer tenant.
- Exportação e envio de e-mail do Portal (POST /portal/exportar, /portal/email) vazam o banco de dados inteiro (animais, produção, sanidade, compras/vendas, financeiro) de todos os clientes para qualquer admin de fazenda.
- Rotas de importação de planilha (animais, genealogia, DairyComp, calendário sanitário) casam registros globalmente por número/nome sem fazenda_id, permitindo sobrescrever cadastro de outro tenant por colisão de número.
- Exclusão em cascata de animal pode apagar histórico reprodutivo/sanitário de outra fazenda quando o número do animal colide entre tenants.
- Padrão sistêmico recorrente (~20 ocorrências): rotas de escrita usam a dependência de leitura tolerante a fazenda_id nulo em vez da dependência de escrita, e checagens de posse que toleram fazenda_id=None em contextos onde isso não deveria ocorrer — RH (folha, contratos, pessoas), sanidade, farmácia, estoque, financeiro.
- Cinco pontos de e-mail/template HTML sem escapar texto de usuário (injeção de HTML em e-mail, e um gerador de contrato com autoescape desligado).

Achados detalhados

1. Banco sem tranca — isolamento por fazenda

Média

Criação de animal grava fazenda_id da dependência tolerante

backend/fazenda/api/routers/cadastro/animais.py:238-262

POST /cadastro/animais usa `get_fazenda_atual_id` (pode ser `None`) em vez de `get_fazenda_id_escrita`. Com token legado/sem fid, cria Animal com `fazenda_id=NULL` em vez de recusar com 409 — registro órfão invisível a qualquer consulta por fazenda.

```
animal = Animal(numero=numero, ativo=dados.data_baixa is None, fazenda_id=fazenda_id)
```

Baixa

Cadastro de Raça grava fazenda_id da dependência tolerante

backend/fazenda/api/routers/cadastro/animais.py:601-618

POST /cadastro/racas usa `get_fazenda_atual_id` em vez de `get_fazenda_id_escrita`, ao contrário dos irmãos MotivoBaixa/MotivoVenda no mesmo arquivo (via `_crud_nome_ativo`).

```
obj = Raca(nome=nome, nota=dados.nota, ativo=dados.ativo, fazenda_id=fazenda_id)
```

Baixa

Cadastro de Grau de Sangue grava fazenda_id da dependência tolerante

backend/fazenda/api/routers/cadastro/animais.py:660-677

Mesmo padrão de criar_raca — POST /cadastro/graus-sangue usa `get_fazenda_atual_id` em vez de `get_fazenda_id_escrita`.

Média

Pesagem corporal criada com fazenda_id da dependência tolerante

backend/fazenda/api/routers/producao.py:591-628

POST /producao/pesagens usa `get_fazenda_atual_id`; o irmão direto criar_controles (ControleLeiteiro) no mesmo arquivo usa corretamente `get_fazenda_id_escrita`.

```
registro = PesagemCorporal(fazenda_id=fazenda_id, numero_matriz=entrada.numero_matriz, data_pesagem=dados.data_pesagem)
```

Média

Qualidade do leite criada com fazenda_id da dependência tolerante

backend/fazenda/api/routers/producao.py:958-968

POST /producao/qualidade-leite usa `get_fazenda_atual_id` em vez de `get_fazenda_id_escrita` — mesma classe de bug.

```
registro = QualidadeLeite(**dados.model_dump(), usuario_id=_usuario_id_seguro(user), fazenda_id=fazenda_id)
```

Média

Entrega de leite mensal criada com fazenda_id da dependência tolerante

backend/fazenda/api/routers/producao.py:1075-1097

POST /producao/entrega-leite usa `get_fazenda_atual_id` na criação; a busca do upsert é corretamente filtrada, mas a criação do novo registro não.

```
registro = EntregaLeiteMensal(**dados.model_dump(), usuario_id=_usuario_id_seguro(user), fazenda_id=fazenda_id)
```

Alta**Relatório de BST agrega Sanidade e Animal de todas as fazendas**

backend/fazenda/api/routers/producao.py:1997-2016

GET /producao/relatorio-bst não recebe fazenda_id e não filtra as consultas. Qualquer usuário autenticado de qualquer fazenda com o módulo 'produtivo' recebe doses de BST, responsável e categoria de animais de TODAS as fazendas do sistema. Todo o resto do arquivo filtra por fazenda_id; esta rota omite o parâmetro por completo.

```
def relatorio_bst(session: Session = Depends(get_session)) -> dict:
    animais_por_numero = {a.numero: a for a in session.exec(select(Animal)).all()}
    aplicacoes = [s for s in session.exec(select(Sanidade)).all() if s.atividade == "BST" ...]
```

Crítica**Ajuste de intervalo de BST sobrescreve parâmetro global ou de outra fazenda**

backend/fazenda/api/routers/producao.py:2024-2089

POST /producao/bst/ajustar-proxima-aplicacao não tem fazenda_id algum. Agrega Sanidade de todas as fazendas e sobrescreve ParametroFazenda (chave 'intervalo_bst'/'bst_ajuste_ancora_data') sem `.where(fazenda_id==...)` — o `.first()` pode pegar o registro global (fazenda_id NULL, afetando todas as fazendas sem override próprio) ou de outro tenant. Gate é só exigir_admin (admin da própria fazenda), não dono da plataforma. Qualquer admin de qualquer fazenda pode sobrescrever silenciosamente o parâmetro de BST de outra fazenda ou o padrão global.

```
aplicacoes = [s for s in session.exec(select(Sanidade)).all() if s.atividade == "BST"]
...
linha_intervalo = session.exec(select(ParametroFazenda).where(ParametroFazenda.chave ==
"intervalo_bst")).first()
linha_intervalo.valor = str(novo_intervalo)
```

Média**Peso corporal na agenda veterinária lido sem filtro de fazenda**

backend/fazenda/api/routers/reproducao.py:460-499

GET /reproducao/agenda-veterinario: todas as outras subconsultas do handler (animais, serviços, partos, secagens, lotes) são filtradas por fazenda_id; a leitura de PesagemCorporal não é. numero_matriz é texto livre sem FK — se colidir entre fazendas, o 'peso mais recente' mostrado pode vir de outro tenant.

```
for p in session.exec(select(PesagemCorporal)).all():
    ... # sem filtro de fazenda_id
```

Média**Adicionar animais a protocolo IATF grava fazenda_id da dependência tolerante**

backend/fazenda/api/routers/reproducao.py:2276-2345

POST /reproducao/protocolo-iatf/{lançamento_id}/animais cria ProtocoloIatfAplicacao com fazenda_id de get_fazenda_atual_id; o irmão direto lancar_protocolo_iatf (lançamento inicial) usa corretamente get_fazenda_id_escrita.

```
session.add(ProtocoloIatfAplicacao(lancamento_id=lançamento_id, numero_matriz=numero, dia=dias, ...,
fazenda_id=fazenda_id))
```

Baixa**Cadastros financeiros (conta corrente, centro de custo, plano de contas) criados com fazenda_id da dependência tolerante**

backend/fazenda/api/routers/financeiro.py:1196,1316,1377-1428; cartao_credito.py:118

criar_conta_corrente, criar_centro_custo, criar_conta_gerencia e a fábrica _crud_nome_ativo_financeiro (tipos-documento/formas-pagamento/classificações) usam get_fazenda_atual_id em vez de get_fazenda_id_escrita, ao contrário de criar_lancamento/criar_pedido/etc no mesmo arquivo. Severidade baixa por serem linhas de catálogo, não transações financeiras.

Baixa**Método de serviço reprodutivo criado/editado com fazenda_id da dependência tolerante**

backend/fazenda/api/routers/cadastro/servicos.py:102-137

criar_metodo_servico/atualizar_metodo_servico usam get_fazenda_atual_id; o restante dos cadastros 'nome+ativo' do sistema já usa get_fazenda_id_escrita via _crud_nome_ativo. Severidade baixa: vocabulário de serviço reprodutivo, não dado financeiro/RH.

Alta**Protocolos IATF/indução concluídos listados sem filtro de fazenda**

backend/fazenda/api/routers/agenda.py:2464-2518 (GET /protocolo-iatf/concluidos e /protocolo-inducao-lactacao/concluidos)

Nenhuma das duas rotas recebe fazenda_id. Qualquer usuário autenticado recebe os protocolos concluídos (números de animal, nomes de protocolo, datas) de TODAS as fazendas.

```
aplicacoes = session.exec(select(ProtocoloIatfAplicacao).where(ProtocoloIatfAplicacao.realizada == True)).all()
```

Média**Última aplicação de produto no calendário sanitário lida sem filtro de fazenda**

backend/fazenda/api/routers/sanidade.py:664-681

_ultimo_evento_por_produto (usada por listar/criar/atualizar calendário) não filtra por fazenda — a data/id de 'última aplicação' exibida no calendário de uma fazenda pode vir de outro tenant que usa produto com o mesmo nome.

Média**Rotas de escrita em sanidade criam registros com fazenda_id da dependência tolerante (6 rotas)**

backend/fazenda/api/routers/sanidade.py:787-924,1659-1691; cadastro/sanitario.py:360-370,512-530,608-626

criar_calendario, criar_cronograma_manual, registrar_colostragem, POST /agendamentos-pesagem, POST /eventos-sanitarios e POST /exames usam get_fazenda_atual_id em vez de get_fazenda_id_escrita — mesma classe de bug 'orfanização silenciosa' documentada no próprio código como causa raiz de um incidente anterior ('D6 sumindo da Agenda').

Média**Migração em massa de dose de protocolo sanitário sem gate de papel e com filtro de fazenda tolerante**

backend/fazenda/api/routers/cadastro/protocolos_sanitarios.py:251-336 (POST /protocolos-sanitarios/dose-migrar)

Sem dependência de papel (qualquer usuário com módulo sanitário pode disparar) e usa get_fazenda_atual_id para uma escrita em massa (`confirmar=True` reescreve modo_dose/unidade/dose_referencia_kg de toda ProtocoloSanitarioEtapa correspondente). Se fazenda_id resolver None, o filtro cai por inteiro e a reescrita atinge TODAS as fazendas de uma vez.

Baixa**Restaurar catálogo de princípios ativos sem gate de papel nem de fazenda**

backend/fazenda/api/routers/cadastro/sanitario.py:302-311 (POST /principios-ativos/restaurar-catalogo)

Sem dependência de fazenda e sem checagem de admin — qualquer usuário comum com acesso ao módulo sanitário pode disparar uma reescrita (add-missing, idempotente) do catálogo GLOBAL de princípios ativos.

Média**Cabeçalho da dieta vaza dados entre fazendas no comparativo**

backend/fazenda/api/routers/alimentacao.py:1761-1802, 1847-1879 (apresentacao_dieta, comparativo_dieta)

O DietaLancamento em si é buscado sem checagem de posse; só os subitens são filtrados por fazenda (voltam vazios entre tenants). O cabeçalho (lote, responsável, observação, datas) de uma dieta de outra fazenda ainda vaza na resposta.

Crítica**Importação de animais por CSV edita/cria Animal globalmente por número, sem fazenda_id**

backend/fazenda/api/routers/importar.py:487-544 (POST /importar/animais_cadastro)

Sem dependência de fazenda_id/usuário no handler. Animal é casado GLOBALMENTE por 'numero' (numeração pequena, tipicamente sequencial por fazenda — alta chance de colisão entre tenants); animais novos são criados com fazenda_id=None. Uma fazenda importando seu próprio rebanho pode sobrescrever silenciosamente o cadastro de um animal de outra fazenda se os números coincidirem.

```
animal = session.exec(select(Animal).where(Animal.numero == numero)).first()
if not animal:
    animal = Animal(numero=numero, ativo=True) # sem fazenda_id
```

Alta**Importação de genealogia edita Animal de qualquer fazenda por número**

backend/fazenda/api/routers/importar.py:547-595 (POST /importar/animais_genealogia)

Mesma causa raiz do achado anterior — sem filtro de fazenda e sem dependência de auth-fazenda no handler, grava pai/mãe em qualquer Animal cujo número coincida.

Crítica**Importação DairyComp cria Animal e Parto sem fazenda_id**

backend/fazenda/api/routers/importar.py:658-729 (POST /importar/dairycomp)

Mesmo padrão de busca global por 'numero' sem filtro de fazenda, e criação de Animal/Parto com fazenda_id=None. Uma fazenda importando seu export DairyComp pode alterar a data de nascimento de animal de outro tenant ou anexar um Parto a ele.

```
animal = session.exec(select(Animal).where(Animal.numero == numero)).first()
if not animal:
    animal = Animal(numero=numero, data_nasc=data_nasc, ativo=True) # sem fazenda_id
...
session.add(Parto(animal_id=animal.id, ...)) # sem fazenda_id
```

Alta**Importação de calendário sanitário cria/edita catálogo global sem fazenda_id**

backend/fazenda/api/routers/importar.py:732-813 (POST /importar/calendario_sanitario)

EventoSanitario/Doenca (cadastros por fazenda) são buscados por nome sem filtro de fazenda e criados sem fazenda_id quando ausentes, apesar de ambos terem restrição única (nome, fazenda_id) que pressupõe todo registro real ser escapado.

Média**Criação de Lida/Protocolo customizado usa fazenda_id da dependência tolerante**

backend/fazenda/api/routers/cadastro/lida.py:109-121; cadastro/protocolos_customizados.py:108-121

Ambas as rotas de criação usam get_fazenda_atual_id em vez de get_fazenda_id_escrita, ao contrário de todo o resto do módulo — no caso-limite documentado (token legado/ambíguo) cria registro órfão (fazenda_id=NULL) em vez de recusar com 409.

Crítica**Exportação/e-mail do Portal vaza dados de TODAS as fazendas**

backend/fazenda/api/routers/portal.py:282-361, 417-448, 476 (POST /portal/exportar, POST /portal/email)

_executar_exportacao monta o ZIP com 10 modelos (Animal, Servico, Parto, ControleLeiteiro, Estoque, MovimentoEstoque, Sanidade, CompraAnimal, VendaAnimal, ContaGerencial) SEM filtro de fazenda_id algum — nem a rota nem a função recebem fazenda_id. enviar_email_portal também chama dre()/rmca()/custo_litro_leite() como funções Python puras, sem fazenda_id: como esses parâmetros são Depends(get_fazenda_atual_id), a chamada direta deixa fazenda_id vinculado ao próprio objeto Depends(...), que fazenda_id_seguro() converte para None — desligando todo filtro por fazenda dentro deles. A rota exige só 'user.papel != admin' → 403 — ou seja, qualquer admin de QUALQUER fazenda-cliente pode se enviar por e-mail um ZIP/CSV com os dados de animais, produção, sanidade, compras/vendas e o livro financeiro de TODOS os outros clientes da plataforma.

```
query = select(cfg["model"])
...
linhas = [row.model_dump(mode="json") for row in session.exec(query).all()] # sem fazenda_id, em 10 tabelas
...
resultado = dre(data_inicio=..., data_fim=..., centro_custo=None, regime="competencia", session=session) # fazenda_id não passado
```

Alta**Estoque de sêmen vaza entre fazendas em dois relatórios**

backend/fazenda/api/routers/relatorios.py:75-82 (GET /relatorios/manejo);
nao_conformidades.py:61 (GET /nao-conformidades/)

select(EstoqueSemen) sem filtro de fazenda_id em ambos os pontos, expondo touro_nome/código/central/tipo/doses de estoque de sêmen de TODAS as fazendas a qualquer usuário autenticado. É exatamente a classe de vazamento que o próprio código em relatorio_acasalamento.py já corrigiu e documenta ('informação comercial de genética dos concorrentes') — a correção não foi replicada nestes dois outros pontos.

Média**Lote lido sem filtro de fazenda em indicadores**

backend/fazenda/api/routers/indicadores.py:313 (GET /indicadores/, POST /relatorio-personalizado)

select(Lote) sem fazenda_id, ao contrário das consultas irmãs (peso_por_animal, aplicacoes_iatf) três linhas acima/abaixo — mistura nomes de lote/configuração de janela DEL de outra fazenda na classificação reprodutiva desta fazenda.

Média**Compra de sêmen/animal e venda de animal criadas com fazenda_id da dependência tolerante**

backend/fazenda/api/routers/compra_semen.py:106-140;
compra_animal.py:92-95;
venda_animal.py:89-92

As três rotas de criação (que também geram ContaGerencial e mexem em estoque/Animal) usam get_fazenda_atual_id em vez de get_fazenda_id_escrita. No caso-limite documentado (token legado sem fid), compra_semen.py também busca EstoqueSemen sem filtro de fazenda quando origem=='estoque', podendo debitar o estoque de outro tenant.

Média**Marcar animal 'a descartar' sem checagem de fazenda quando o token não tem fazenda selecionada**

backend/fazenda/api/routers/baixas.py:73-129 (POST /baixas/a-descartar)

Diferente do irmão POST /baixas/ (que usa get_fazenda_id_escrita), esta rota usa a dependência tolerante; quando resolve None, a busca do Animal por 'numero' fica sem filtro de fazenda algum — números de animal se repetem entre fazendas (confirmado em comentário do próprio upload.py), então isso pode marcar o animal errado (de outra fazenda) para descarte.

Baixa

Padrão sistêmico recorrente: criação usando `get_fazenda_atual_id` em vez de `get_fazenda_id_escrita` (motivos de movimentação, lotes, safra, princípios de genética, alertas de indicador)

backend/fazenda/api/routers/movimentacoes.py:118-234;
lotes.py:220-312; safra.py:63-111;
cadastro/genetica.py:229-278;
alertas_indicador.py:118-139

Mesma classe de bug já descrita: criação de `MotivoMovimentacao/Lote/Safra/EstoqueSemen/AlertaIndicador` com `fazenda_id` potencialmente `NULL` em vez de recusar com 409. O caso do `MotivoMovimentacao` é o mais visível: uma linha órfã (`fazenda_id=NULL`) passa a aparecer como padrão global para TODAS as fazendas na listagem de motivos. O caso do `AlertaIndicador` é o mais sensível: um alerta criado com `fazenda_id=NULL` é avaliado depois sem filtro de fazenda, agregando dados de TODOS os tenants no '`valor_atual`' mostrado a um único usuário.

Baixa

Endpoint de diagnóstico do Telegram sem autenticação vaza lista de chats liberados

backend/fazenda/api/routers/telegram.py:688-696 (GET /telegram/status)

Rota pública, sem nenhuma dependência de autenticação nem proteção no `include_router`. Divulga a `allow-list` completa de `TELEGRAM_ALLOWED_CHAT_IDS` a qualquer um na internet — reconhecimento útil para quem já obteve/adivinhou o segredo do webhook.

Média

Chat do Telegram não mapeado vaza documento financeiro para a fila de aprovação de todas as fazendas

backend/fazenda/api/routers/telegram.py:100-116 (`_fazenda_do_chat`);
aprovacoes.py:61-62

`TELEGRAM_BOT_TOKEN/TELEGRAM_ALLOWED_CHAT_IDS` são globais (não por fazenda). Um chat autorizado em `TELEGRAM_ALLOWED_CHAT_IDS` mas ausente de `TELEGRAM_CHAT_FAZENDA` gera um `LancamentoPendente` com `fazenda_id=None`, que GET /aprovacoes mostra e permite aprovar/editar em TODAS as fazendas (`'fazenda_id.in_((fazenda_id, None))'`). Um único esquecimento de configuração ao integrar um novo chat-cliente vaza a entrada de documentos financeiros (nota, valor, contraparte) dessa fazenda para os aprovadores de todas as outras, até o mapeamento ser corrigido.

3. IDOR / controle de acesso quebrado

Alta

Editar Serviço tolera `fazenda_id` nulo e permite editar registro de outra fazenda

backend/fazenda/api/routers/reproducao.py:856-880

PUT /reproducao/servicos/{servico_id}: a checagem `'servico.fazenda_id not in (None, fazenda_id)'` tolera um Serviço órfão (`fazenda_id=NULL`). 'servico' está listado em `_TABELAS_SEM_PAI` na migração 029227481e9e_backfill_fazenda_id_nulo.py, cujo próprio relatório documenta registros que continuam `NULL` após a migração em instalações com 2+ fazendas. Nessa condição, qualquer usuário de qualquer fazenda pode editar diagnóstico/perda de prenhez desse registro só enumerando o id.

```
servico = session.get(Servico, servico_id)
if not servico or (fazenda_id is not None and servico.fazenda_id not in (None, fazenda_id)):
    raise HTTPException(404, ...)
```

Alta

Editar Parto tolera `fazenda_id` nulo — mesmo padrão do Serviço

backend/fazenda/api/routers/reproducao.py:1193-1214

PUT /reproducao/partos/{parto_id} tem a mesma checagem tolerante a `fazenda_id=NULL` de `atualizar_servico` ('parto' também está em `_TABELAS_SEM_PAI`). O irmão `atualizar_secagem`, no mesmo arquivo, já foi corrigido para uma checagem estrita (`'!='`) documentando o bug G5 — a correção não foi replicada aqui.

```
parto = session.get(Parto, parto_id)
if not parto or (fazenda_id is not None and parto.fazenda_id not in (None, fazenda_id)):
    raise HTTPException(404, ...)
```

Alta**Vincular evento sanitário/reprodutivo a lançamento sem checagem de posse**

backend/fazenda/api/routers/financeiro.py:1684-1699

POST /financeiro/vincular-evento-sanitario-reprodutivo não recebe fazenda_id algum (o irmão candidatos_vinculo_sanitario_reprodutivo logo acima recebe). Qualquer usuário autenticado de qualquer fazenda pode passar ids de Sanidade/ExameResultado/Servico de outro tenant e vinculá-los a um numero_lancamento à escolha, sem nenhuma checagem de posse antes do write.

```
obj = session.get(modelo, item_id)
if obj:
    obj.numero_lancamento_vinculado = dados.numero_lancamento
```

Alta**Recibo por número de lançamento vaza dados e envia e-mail para qualquer tenant**

backend/fazenda/api/routers/financeiro.py:4158-4207

GET .../destinatario-recibo e POST .../recibo/enviar não recebem fazenda_id. numero_lancamento é sequencial e global (LC-{ano}-{00001,...}, sem filtro por fazenda em _proximo_numero_lancamento) — facilmente enumerável entre tenants. O GET vaza nome/e-mail do fornecedor/cliente de outra fazenda; o POST envia e-mail (com nome do fornecedor e valor de outra fazenda) para um destinatário arbitrário controlado pelo atacante, usando o remetente da CowData.

```
conta = session.exec(select(ContaGerencial).where(ContaGerencial.numero_lancamento ==
numero_lancamento)).first()
# sem checagem de fazenda_id em nenhuma das duas rotas
```

Alta**Webhook de cobrança do Banco do Brasil não valida o segredo do path**

backend/fazenda/api/routers/cobranca.py:102-131

POST /cobranca/webhook/bb/{secret}: o parâmetro `secret` é capturado mas NUNCA comparado a nenhum valor configurado — só se checa se BB_CLIENT_ID está definido. O payload (`status`) é confiado diretamente, sem reconfirmar via API do BB. zapsign.py e asaas.py fazem a comparação/reconfirmação corretas no mesmo repositório, confirmando que este é um gap real. Qualquer um que descubra a URL pode marcar boletos/PIX como pagos.

```
async def cobranca_webhook_bb(secret: str, request: Request, session=...) -> dict:
    if not settings.bb_client_id:
        raise HTTPException(403, ...)
    payload = await request.json()
    if payload.get("status") in ("CONCLUIDA", "paga"): pix.status = "paga" # secret nunca comparado
```

Média**Cartão de crédito aceita conta bancária de outra fazenda sem checar posse**

backend/fazenda/api/routers/cartao_credito.py:118-150

criar_cartao/atualizar_cartao só checam que ContaCorrente existe, não que pertence à fazenda do chamador (ao contrário de _cartao_ou_404/_fatura_ou_404 no mesmo arquivo). pagar_fatura_cartao depois expõe banco/agência/número da conta de outro tenant na resposta e grava esse texto no próprio lançamento do atacante.

```
if dados.conta_bancaria_id and not session.get(ContaCorrente, dados.conta_bancaria_id):
    raise HTTPException(404, ...) # sem checar fazenda_id
```

Alta**Folha de pagamento: PUT/DELETE tolera token sem fazenda selecionada (17 rotas)**

backend/fazenda/api/routers/cadastro/rh_folha.py — 17 ocorrências
(ex.: 535-543 PUT /folha-pagamento/{id}, 612-620 DELETE idem, 762-827 guias, 944-1014 férias, 1131-1203 13º, 1619-1650 rescisões, 2151-2542 vales/comprovantes)

get_fazenda_atual_id retorna None sempre que o token não carrega "fid" — estado válido e normal para usuário vinculado a 2+ fazendas que ainda não chamou /auth/selecionar-fazenda (token útil por até 12h, ou 90 dias com 'manter conectado'). Nesse estado, `fazenda\_id is not None and obj.fazenda\_id != fazenda\_id` vira sempre False — a checagem de posse é pulada por inteiro em TODAS as 17 rotas de escrita/leitura de anexo listadas, permitindo ler/editar/excluir folha, férias, 13º, rescisão, vale e comprovantes de QUALQUER fazenda só incrementando o id. Rotas de criação no mesmo arquivo já usam corretamente get_fazenda_id_escrita, provando que é uma inconsistência corrigível, não um design deliberado.

```
registro = session.get(FolhaPagamento, registro_id)
if not registro or (fazenda_id is not None and registro.fazenda_id != fazenda_id):
    raise HTTPException(404, ...)
```

Alta**Empreitadas/contratos/diárias/vale-avulso: mesmo gap de PUT/DELETE (11 rotas)**

backend/fazenda/api/routers/cadastro/rh_contratos.py — 11 ocorrências
(ex.: 413-504 parcelas de empreitada, 639-711 contratos, 1031-1088 diárias, 1785-1793 vale-avulso)

Mesmo mecanismo e pré-condição da Finding 1 (rh_folha.py). Um portador de token válido mas sem fazenda selecionada pode encerrar/editar/excluir empreitada, contrato, diária e vale-avulso de qualquer fazenda por id. As rotas de criação no mesmo arquivo usam corretamente get_fazenda_id_escrita com checagem incondicional.

```
diaria = session.get(Diaria, diaria_id)
if not diaria or (fazenda_id is not None and diaria.fazenda_id != fazenda_id):
    raise HTTPException(404, ...)
```

Alta**Dados pessoais (Pessoa) e anexos de documento (RG/CPF/holerite): mesmo gap de posse**

backend/fazenda/api/routers/cadastro/pessoas.py:411-463, 514-606

PUT/DELETE /cadastro/pessoas/{id} e upload/download/exclusão de anexos (PessoaAnexo) usam a mesma checagem tolerante a fazenda_id=None das duas findings acima. Um usuário com token de fazenda não selecionada e permissão de 'parametros' pode baixar/excluir RG/CPF/holerite de qualquer fazenda, ou editar salário_base/CPF/endereço de qualquer pessoa entre tenants.

```
anexo = session.get(PessoaAnexo, anexo_id)
if not anexo or (fazenda_id is not None and anexo.fazenda_id != fazenda_id):
    raise HTTPException(404, ...)
```

Média**Dados de RH acessíveis/editáveis em modo suporte apesar do bloqueio pretendido**

backend/fazenda/api/routers/cadastro/pessoas.py (todo o arquivo);
main.py:413-458
(_PREFIXOS_RH_MODAL_SUPORTE)

Nenhum caminho /cadastro/pessoas... aparece na lista que _bloquear_modal_suporte usa para bloquear escrita e leitura abaixo do sigilo 'total' durante uma sessão de suporte CowData. Mas salário_base alimenta diretamente o cálculo de férias/13º/rescisão, e PessoaAnexo inclui RG/CPF/holerite — dados tão sensíveis quanto o resto de RH que já é bloqueado. Uma sessão de suporte no menor nível de sigilo pode hoje ler esses anexos e, diferente de todo o resto de RH, também ESCREVER salário_base/CPF/endereço via PUT, distorcendo silenciosamente cálculos futuros de folha.

Crítica**Confirmar protocolo IATF na Agenda sem checagem de fazenda (write cross-tenant)**

backend/fazenda/api/routers/agenda
 .py:2025-2031, 2159-2165, 2260-2269
 (dispatch em POST
 /agenda/realizados)

A busca de ProtocoloIatfAplicacao por (data_prevista, dia, realizada=False) não filtra por fazenda_id. evento_id vem direto do corpo da requisição. Datas D0/D7/D9/D11 são fixas e lancamento_id é inteiro pequeno sequencial — qualquer usuário autenticado de QUALQUER fazenda pode confirmar (ou, via DELETE, desconfirmar) protocolos IATF de outro tenant, gravando Sanidade falsa referenciando o animal da vítima e consumindo o próprio estoque do atacante.

```
aplicacoes = session.exec(
select(ProtocoloIatfAplicacao).where(
ProtocoloIatfAplicacao.data_prevista == data_prevista,
ProtocoloIatfAplicacao.dia == dia,
ProtocoloIatfAplicacao.realizada == False,
)
).all() # sem fazenda_id
```

Crítica**Confirmar protocolo customizado / lida sem checagem de fazenda**

backend/fazenda/rules/protocolo_customizado.py:127-133; backend/fazenda/rules/lida.py:106-112; dispatch em agenda.py:2270-2277

Mesmo bug do item acima aplicado aos protocolos customizados e à 'lida' — marcar_realizado() seleciona por lancamento_id+dia sem qualquer filtro de fazenda. O caminho de lida também gera baixa de estoque indevida usando o estoque do atacante para uma tarefa de outra fazenda.

Crítica**Confirmar aplicação de protocolo sanitário / vacina pré-parto por id sequencial adivinhável**

backend/fazenda/api/routers/agenda
 .py:1857-1966 (dispatch em
 2299-2307)

protocolo_sanitario_{id} e aplic_agendada_{id} usam inteiros pequenos e sequenciais, sem nenhuma checagem de fazenda em session.get(...). O caso 'vacina_pre_parto_{numero}_{data}' não tem filtro de fazenda nenhum. Qualquer usuário pode adivinhar o id de outro tenant e gravar uma aplicação sanitária falsa nesse animal.

```
aplicacao_id = int(evento_id.removeprefix("protocolo_sanitario_"))
aplicacao = session.get(ProtocoloSanitarioAplicacao, aplicacao_id) # sem checagem de fazenda
```

Alta**Desmarcar protocolo confirmado (DELETE) tem o mesmo gap dos itens acima**

backend/fazenda/api/routers/agenda
 .py:2550-2570 (dispatch em _desmarcar_protocolo_iatf_realizado:2439-2461, _desmarcar_protocolo_inducao_realizado:2224-2241, protocolo_customizado.desmarcar_realizado:146-159, lida.desmarcar_realizado:141-160)

DELETE /agenda/realizados/{evento_id} é o espelho dos achados acima: reverte um tratamento CONFIRMADO de outro tenant de volta para pendente, apagando silenciosamente o registro de conformidade da vítima.

Alta**Marcar animal inapto para BST por número, sem checagem de fazenda**

backend/fazenda/api/routers/agenda
 .py:2416-2436 (POST
 /bst/marcar-inapta)

Animal é buscado globalmente por 'numero' (campo de entrada do cliente), sem fazenda_id algum. Qualquer usuário pode alternar as flags de elegibilidade a BST de um animal de outra fazenda.

```
animal = session.exec(select(Animal).where(Animal.numero == numero)).first()
animal.excluir_bst = dados.inapta
```

Alta**Cadastrar item preventivo no calendário aceita evento/regra de outra fazenda**

```
backend/fazenda/api/routers/sanidade.py:1046-1106 (POST /sanidade/calendario/cadastrar-preventivo)
```

EventoSanitario e CalendarioSanitario são cadastros por fazenda (não catálogo global), mas são buscados por id sem checar fazenda_id — só se confere que o calendario_id aponta pro mesmo evento_id (ambos inteiros pequenos e sequenciais). A resposta ecoa nome do protocolo, doença, produto, dosagem, veterinário e observação da vítima, podendo também semear uma nova regra na fazenda do atacante copiada dos padrões da vítima.

Média**PUT/DELETE em sanidade e protocolos sanitários toleram fazenda_id nulo (12 rotas), incluindo o catálogo global de princípio ativo/doença**

```
backend/fazenda/api/routers/sanidade.py (editar/excluir aplicação, calendário, exame, cura);
cadastro/protocolos_sanitarios.py (atualizar/excluir protocolo sanitário/indução/IATF);
cadastro/_comum.py:88-98 (_crud_nome_ativo, usado por Princípio ativo/Doença)
```

Mesmo padrão sistêmico de fazenda_id=None tolerado em checagens de posse já descrito para RH — aqui aplicado a protocolos sanitários. O caso de _comum.py é o mais grave: Princípio ativo/Doença são o catálogo global União (fazenda OU nula) por design — em estado de token ambíguo, um usuário comum poderia editar a própria linha GLOBAL compartilhada por todas as fazendas, não só uma linha de outro tenant.

Alta**Exclusão de marca/indicação de medicamento não recebeu o mesmo fix do irmão PUT**

```
backend/fazenda/api/routers/farmacia.py:266-276, 408-418, 749-766
```

atualizar_marca/atualizar_indicacao já foram corrigidos (comentário no código documenta o bug: token sem 'fid' editava direto o registro GLOBAL) trocando para get_fazenda_id_escrita. Os DELETES irmãos (excluir_marca, excluir_indicacao, .../personalizar) continuam usando get_fazenda_atual_id — correção assimétrica que deixa o mesmo bug aberto para exclusão.

Alta**Renomear produto da tabela nutricional sem checagem de fazenda**

```
backend/fazenda/api/routers/alimentacao.py:1261-1274 (PUT /alimentacao/tabela-nutricional/produtos/{id})
```

Sem fazenda_id algum, ao contrário de todas as rotas irmãs do mesmo bloco. Qualquer usuário de qualquer fazenda pode renomear o produto de outro tenant só adivinhando/incrementando o id.

```
produto = session.get(TabelaNutricionalProduto, produto_id)
if not produto: raise HTTPException(404, ...)
produto.nome = nome # sem checar fazenda_id
```

Alta**Excluir produto da tabela nutricional sem checagem de fazenda**

```
backend/fazenda/api/routers/alimentacao.py:1277-1287 (DELETE .../produtos/{id})
```

Mesmo gap do achado anterior, mas destrutivo: exclusão permanente de produto e valores nutricionais de outra fazenda.

Crítica**Encerrar dieta de outra fazenda sem checagem alguma**

backend/fazenda/api/routers/alimentacao.py:1809-1819 (PUT
/alimentacao/dietas/{id}/encerrar)

Nenhuma dependência de fazenda. Qualquer usuário autenticado de qualquer fazenda pode encerrar a dieta ATIVA de outro tenant só adivinhando dieta_id, interrompendo a baixa automática de estoque e a apuração da fazenda vítima.

```
dieta = session.get(DietaLancamento, dieta_id)
if not dieta: raise HTTPException(404, ...)
dieta.data_efetivo_encerramento = dados.data_efetivo_encerramento # sem checar fazenda_id
```

Alta**Registrar alimentação real em dieta de outra fazenda**

backend/fazenda/api/routers/alimentacao.py:1827-1844 (POST
/alimentacao/dietas/{id}/real)

fazenda_id do novo registro é resolvido corretamente, mas dieta_id (referenciado) nunca é checado contra a fazenda do chamador — qualquer usuário pode anexar registros de 'consumo real' à dieta de outro tenant.

Crítica**Exclusão em cascata cruza fazendas quando o número do animal colide**

backend/fazenda/api/routers/exclusoes.py:502-607 (usado por POST
/exclusoes/confirmar e
/pendentes/{id}/aprovar)

A busca do Animal é corretamente escopada por fazenda, mas as buscas seguintes de Serviço/Parto/ControleLeiteiro/Sanidade (e AgendaManual, para retenção de placenta) filtram só por numero_matriz, SEM fazenda_id, e são de fato deletadas por _excluir_alvos_em_ordem. Números de animal são pequenos e reutilizados entre fazendas diferentes — um admin excluindo seu próprio animal '464' também apaga o histórico reprodutivo/produtivo/sanitário do animal '464' de OUTRA fazenda, se os números coincidirem. Bug real disparável em uso normal, não só por má-fé.

```
servicos = session.exec(select(Servico).where(Servico.numero_matriz == id_)).all() # sem fazenda_id
partos = session.exec(select(Parto).where(Parto.numero_matriz == id_)).all() # sem fazenda_id
...
return impacto, [animal, *servicos, *partos, *controles, *sanidades, ...] # depois deletados
```

Crítica**Autopromoção a dono-equivalente via PUT /auth/preferencias — CORRIGIDO NESTA SESSÃO**

backend/fazenda/api/routers/auth.py:518-527 (salvar_preferencias)

eh_email_dono_equivalente() concede acesso total de dono (bypassa exigir_dono, exigir_admin_ou_dono e tudo que depende deles, incluindo o Painel CowData inteiro) a qualquer usuário cujo e-mail esteja em EMAILS_DONO_EQUIVALENTE = {EMAIL_DONO, "alexandrescarpazoo@yahoo.com.br"}. Esta rota só tratava o caso EMAIL_DONO — o segundo e-mail dono-equivalente nunca era checado. PUT /auth/preferencias exige só um login ativo (get_current_user), qualquer papel, qualquer fazenda. Qualquer operador podia virar dono-equivalente enviando esse e-mail uma única vez, sem NENHUMA checagem de admin nesse ramo. CORRIGIDO nesta sessão: a condição agora usa eh_email_dono_equivalente(...) e rejeita com 403 qualquer e-mail dono-equivalente que não seja literalmente EMAIL_DONO, preservando as travas de admin/dono-já-existente só para o caminho legítimo de auto-atendimento do EMAIL_DONO. Teste de regressão adicionado em tests/test_auth_preferencias.py.

```
novo_email = EMAIL_DONO if dados.reivindicar_proprietario else dados.email
if novo_email and novo_email.lower() == EMAIL_DONO: # ANTES: só comparava com EMAIL_DONO
if user.papel != "admin": raise HTTPException(403, ...)
...
```

Alta**Painel CowData > Usuários permite criar/editar login com e-mail dono-equivalente — CORRIGIDO NESTA SESSÃO**

backend/fazenda/api/routers/painel_cowdata_usuarios.py:101-165
(criar_usuario_da_fazenda / editar_usuario_da_fazenda)

Ambas as rotas são gated só por `exigir_area_painel_cowdata("cadastros")` — área de baixo privilégio, não `exigir_dono` — e nenhuma validava o e-mail contra `EMAILS_DONO_EQUIVALENTE`. Um funcionário do CowData com acesso apenas a 'cadastros' podia criar/editar um login de QUALQUER fazenda-cliente com `email="alexandrescarpazoo@yahoo.com.br"` e senha à escolha, produzindo uma conta com acesso total de dono — escalção de uma permissão estreita para controle total da plataforma. CORRIGIDO nesta sessão: ambas as rotas agora rejeitam com 403 qualquer e-mail dono-equivalente. Testes de regressão adicionados em `tests/test_painel_cowdata_usuarios.py`.

```

novo = Usuario(..., email=(dados.email or "").strip() or None) # sem checagem de dono-equivalente

```

Alta**Encerrar sessão de suporte não é aplicado pelo middleware (revogação sem efeito)**

backend/fazenda/api/routers/cofre_acesso.py:253-266;
fazenda/models/cofre_acesso.py:41

`encerrar_sessao` só marca uma flag no banco e grava auditoria. O token assinado emitido em `solicitar_acesso/aprovar_pedido` é validado de forma totalmente stateless (`_validar_token_payload`, `get_current_user`, `_bloquear_modos_suporte` em `main.py`) — nenhum desses consulta `SessaoAcessoSuporte` para saber se a sessão foi encerrada. `grep` confirma que `SessaoAcessoSuporte` só é referenciado dentro de `cofre_acesso.py`, nunca no caminho de `auth/middleware`. Um token de suporte continua com acesso total (exceto os prefixos bloqueados) por até 30 minutos mesmo após ser 'encerrado', contradizendo o próprio comentário do modelo.

Alta**Catálogo global de touros (genética) editável/excluível por qualquer funcionário**

backend/fazenda/api/routers/cadastro/genetica.py:403-509
(POST/PUT/DELETE /touros...)

Touro é deliberadamente um catálogo global (sem `fazenda_id`) usado por todas as fazendas. As rotas de escrita não têm NENHUM gate de papel além do `exigir_modulo("parametros")` do roteador (permissão comum de funcionário, não admin/dono). Qualquer funcionário comum com o módulo 'parametros' de qualquer fazenda-cliente pode reescrever, excluir ou reimportar em massa o catálogo de touros usado por TODOS os outros clientes, sem forma de escopar ou desfazer por tenant.

```

@router.delete("/touros/{touro_id}")
def excluir_touro(touro_id: int, session=...) -> dict:
    t = session.get(Touro, touro_id)
    session.delete(t) # sem checagem de papel além de exigir_modulo("parametros")

```

Média**Atualizar chamado de suporte sem gate de papel**

backend/fazenda/api/routers/chamados.py:70-88 (PUT /chamados/{id})

Chamado representa um ticket de suporte aberto pela fazenda para o administrador/CowData; o campo 'resposta' representa a resposta oficial do suporte. Qualquer usuário autenticado com o módulo financeiro (sem `exigir_admin/exigir_dono`) pode fechar o ticket e fabricar uma 'resposta', se passando pelo suporte oficial.

Alta**Upload de documentos/fotos: extensão de arquivo não sanitizada permite escrever fora do prefixo esperado no Storage**

backend/fazenda/api/routers/documentos.py:68-97; fotos.py:102-132; fazenda/rules/supabase_storage.py:74-75

A extensão é tudo depois do último '.' do nome de arquivo enviado pelo cliente, sem rejeitar '/' ou '..', e é concatenada direto na URL REST do Supabase Storage sem sanitização (a função nome_seguro_storage() existe no mesmo módulo mas não é chamada por esses dois uploads). Um nome como 'evil.png/../../outra-fazenda/arquivo.pdf' produz uma extensão contendo '../../', potencialmente escrevendo/sobrescrevendo objetos fora do prefixo 'fazenda-{id}/...' pretendido, inclusive de outro tenant. O mesmo caminho também deixa o cliente definir o Content-Type salvo, servido depois com Content-Disposition: inline — um tipo como text/html/image/svg+xml servido inline é um risco de XSS armazenado caso a rota de download seja aberta na mesma origem autenticada.

```
nome_original = file.filename or "arquivo"
extensao = f"{nome_original.rsplit('.', 1)[-1].lower()}" if "." in nome_original else ""
...
caminho = f"{prefixo_pasta}/{prefixo_nome}{seq:04d}{extensao}" # sem sanitização
```

5. Inputs sem tratamento (XSS)

Média**E-mail do Portal injeta HTML não escapado do corpo digitado pelo usuário**

backend/fazenda/api/routers/portal.py:286-289 (POST /portal/email)

dados.corpo é texto livre de qualquer usuário autenticado (não exige admin) e é interpolado cru em HTML enviado por e-mail via Resend a outros usuários reais do sistema. Permite injetar links ocultos, imagens/pixels de rastreamento ou spoofing de conteúdo dentro de um e-mail que aparenta vir da plataforma. Achado confirmado independentemente por dois agentes da auditoria (categorias 1 e 5).

```
corpo_html = f"<p>{dados.corpo}</p>" if dados.corpo else "<p></p>"
```

Alta**Geração de contrato usa Jinja2 sem autoescape — HTML/script do cadastro da fazenda é interpolado cru**

backend/fazenda/rules/contrato_render.py:14, 73; backend/fazenda/templates/contrato_cowdata.html:108-204; backend/fazenda/api/routers/fazendas.py:185-224, 639-663

jinja2.Template() tem autoescape OFF por padrão (confirmado empiricamente:

`Template('<p>{{x}}</p>').render(x='<script>...</script>')` retorna o script intacto). O template do contrato interpola fazenda_nome/endereco/representante_nome/representante_cpf/cidade_foro/estado_foro como {{ }} cru. Esses campos vêm do cadastro da Fazenda (só .strip(), sem validação de HTML) e PODEM SER SOBRESCRITOS DIRETO PELA QUERY STRING de GET /fazendas/{id}/contrato/modelo. A resposta é servida com Content-Disposition: inline (renderização no navegador). O irmão contrato_equipe_render.py já usa Environment(autoescape=select_autoescape(["html"])) corretamente — prova de que o padrão certo é conhecido, só não foi aplicado aqui. Mitigação parcial: o único chamador do frontend força download do arquivo em vez de renderizar inline, mas a rota em si continua sendo um sink de HTML não escapado se acessada diretamente.

```
from jinja2 import Template
...
return Template(arquivo.read_text(encoding="utf-8")).render(**contexto) # autoescape OFF por padrão
```

Média**Recibo por e-mail injeta nome de fornecedor/cliente não escapado**

backend/fazenda/api/routers/financeiro.py:4197-4200

conta.fornecedor_cliente (texto livre digitado ao cadastrar um lançamento, sem exigir admin) é interpolado cru em HTML de e-mail enviado a um destinatário arbitrário digitado no modal de 'enviar recibo'.

```
corpo_html = f"<p>Segue em anexo o recibo do lançamento <b>{numero_lancamento}</b> (<{conta.fornecedor_cliente or '-'>, R$ {conta.valor_total or 0:.2f}</p>"
```

Média

E-mail de diagnóstico reprodutivo injeta campos de texto livre não validados

backend/fazenda/api/routers/reproducao.py:720-729, 856-880

Servico.diagnostico/metodo_diagnostico/diagnostico_reconfirmacao são texto livre sem validação por enum (só um comentário dizendo o que deveria ser). PUT /servicos/{id} faz um `setattr` genérico sobre qualquer campo enviado, sem validar diagnostico/metodo_diagnostico (só motivo_perda_prenhez tem allow-list). Esse valor depois é interpolado cru no corpo HTML do e-mail de diagnóstico — caminho de exploração confirmado ponta a ponta (rota de escrita sem validação → sink de e-mail sem escape).

```
linhas.append(f"<p><b>Método:</b> {servico.metodo_diagnostico}</p>")
```

Baixa

E-mail de redefinição de senha injeta nome do usuário sem escapar (majoritariamente self-XSS)

backend/fazenda/api/routers/auth.py:330-335

user.nome é interpolado cru no e-mail de redefinição de senha, enviado só para o próprio endereço cadastrado do usuário — exploração exigiria que um admin já tivesse cadastrado um nome malicioso para essa pessoa (baixo valor prático), mas ainda é texto não escapado em HTML de e-mail.

```
corpo_html = f"<p>Olá, {user.nome or user.username}!</p>..."
```

4. Chaves expostas

Informativa

Campo secret_key morto em config.py, com valor de aparência insegura

backend/fazenda/config.py:9

secret_key: str = "change-me" é o único lugar do repositório inteiro (varrido inclusive .env, migrações, seeds, CI, docs) onde esse identificador aparece — nenhum código lê settings.secret_key. Não é explorável hoje por ser código morto, mas deveria ser removido (ou receber o mesmo guard de startup que AUTH_SECRET tem) para não induzir um futuro contribuidor a usá-lo pensando que é seguro.

```
secret_key: str = "change-me"
```

Itens incertos (necessitam confirmação humana)

reproducao.py:2290,2363,2768 (ProtocoloIatfLancamento) — Mesma forma de tolerância a `fazenda_id` nulo dos achados confirmados, mas não foi possível confirmar por leitura estática se existem hoje linhas órfãs de `protocolo_iatf_lancamento` no banco de produção. Recomenda-se checar ``SELECT count(*) FROM protocolo_iatf_lancamento WHERE fazenda_id IS NULL`` antes de decidir a prioridade do hardening.

animais.py:359-368,531-644 (BaixaAnimal, CompraAnimal, VendaAnimal, MovimentoLote, OcorrenciaClinica) — Modelos sem coluna `fazenda_id`, consultados só por `numero_animal`. Gap já reconhecido e documentado no próprio docstring do arquivo como pendente de uma futura separação de domínio Fazenda/Empresa — não é um achado novo, mas não deve ser silenciosamente perdido do relatório agregado.

cadastro/rh_vale_item.py (mounting em cadastro/_init_.py:83-86) — Rotas de RH/vale não têm o mesmo bloqueio `bloquear_escrita_contador()` que `financeiro.py/cartao_credito.py` têm. Se um contador tiver também a permissão 'parametros' (não confirmado nesta auditoria estática), poderia escrever em vale de funcionário apesar de ser pensado como só-leitura para esse papel.

aprovacoes.py:60-77,246-250; telegram.py:100-116 (_fazenda_do_chat) — `LancamentoPendente` com `fazenda_id=None` é visível/acionável por qualquer fazenda por design (acomodação para instalação legada), mas em produção multi-tenant real isso depende de `TELEGRAM_CHAT_FAZENDA` cobrir todo chat ativo — não verificável estaticamente; recomenda-se conferência operacional dessa variável de ambiente.

cofre_acesso.py:206-234 (aprovar_pedido/negar_pedido) — Sem checagem de que o aprovador é diferente do solicitante (maker-checker). Impacto real hoje é nulo porque nenhuma fazenda-piloto ativa exige `aprovacao_suporte=True` (conforme comentário do próprio código), mas o controle não tem nenhuma garantia no momento em que uma fazenda ativar essa opção.

main.py:589-678 (_idempotencia) — O cache de idempotência é resolvido por (chave, método, caminho) + `fazenda_id` best-effort, sem vínculo ao usuário nem prova de que o segundo chamador já tinha um token válido. Se `fazenda_id` for `None` (ex.: `POST /auth/login`) e um `Idempotency-Key` for reaproveitado/adivinhado, o cache devolveria a resposta cacheada (incluindo um token de login de outro usuário) sem apresentar credenciais. Exploração depende de o esquema de geração da chave no frontend ser previsível — não avaliado nesta auditoria.

auth.py:36-53 (_rodando_em_producao) — Heurística de detecção de produção (`DATABASE_URL` começa com `postgres://`) casa com o formato real do Railway hoje, mas uma string de conexão diferente (ex.: `postgresql+psycpg://`, um proxy/pooler) faria exigir `segredo_de_producao()` passar silenciosamente. Vale checagem pontual de quem administra a infra.

cadastro/pessoas.py (todo o arquivo) — Dados sensíveis de RH (salário, CPF, RG, anexos pessoais) são protegidos só pelo módulo genérico 'parametros', mais amplamente concedido que 'financeiro'. Pode ser decisão deliberada de produto (Pessoa é cadastro transversal usado por vários módulos) — recomenda-se confirmação humana antes de restringir.

alimentacao.py:1366-1390 (salvar_valores_tabela_nutricional) — `produto_id` do cliente não é checado contra posse antes de criar um novo `TabelaNutricionalValor`, mas o caminho de leitura sempre junta de volta pelo `produto` já filtrado por fazenda do chamador — parece só criar linhas órfãs mortas, não vazamento visível. Recomenda-se confirmar que nenhum relatório/exportação futuro junta essa tabela sem o mesmo filtro.

importar.py:1004-1037 (importar_touros_naab); importar.py:831-1001 — Touro é catálogo global por design (sem `fazenda_id`) — a escrita sem gate de fazenda parece intencional, mas vale confirmar se a amplitude de quem pode chamar (qualquer usuário com módulo 'upload') é realmente a intenção do produto.

fotos.py:238-253 (excluir_foto) — Qualquer usuário da fazenda (não só admin) pode excluir foto de qualquer outro usuário da mesma fazenda — pode ser intencional (conteúdo operacional), mas é inconsistente com `documentos.py`, que restringe `DELETE` a admin explicitamente por ser irreversível.

portal.py:354-361 (delegar_tarefa) — Texto livre do usuário é gravado em `AgendaManual.descricao` sem HTML envolvido no backend — só vira XSS se algum componente do frontend renderizar esse campo via inserção de HTML não escapado; não confirmado nesta auditoria (o agente de frontend não encontrou tal renderização, mas o campo não foi rastreado ponta a ponta).

pedidos.py (link_rastreio) e farmacia.py (link_bula) no frontend — Campos de URL livre sem allow-list de esquema (nem frontend nem backend) — um valor javascript: poderia ser salvo. Impacto prático limitado por exigir usuário já autenticado com permissão de escrita, e por navegadores modernos bloquearem javascript: em target="_blank".

Recomendações priorizadas

P1 — Crítico, corrigir imediatamente

- [JÁ CORRIGIDO nesta sessão] Escalação de privilégio a dono via e-mail dono-equivalente em PUT /auth/preferencias e em Painel CowData > Usuários — testes de regressão adicionados.
- [CORRIGIDO nesta sessão] Corrigir todas as rotas de confirmação/desconfirmação de protocolo na Agenda (agenda.py) para exigir e checar fazenda_id antes de gravar/apagar qualquer Sanidade/aplicação — reaproveitar o helper _lançamento_ou_404/_exigir_da_fazenda já existente na Central de Protocolos.
- [CORRIGIDO nesta sessão] Corrigir POST /portal/exportar e POST /portal/email para filtrar todas as 10 tabelas do EXPORT_CATALOG por fazenda_id e passar fazenda_id explicitamente para dre()/rmca()/custo_litro leite() em vez de chamá-los como função pura.
- [CORRIGIDO nesta sessão] Corrigir as rotas de importação de planilha (importar.py: animais_cadastro, animais_genealogia, dairycomp, calendario_sanitario) para resolver fazenda_id via get_fazenda_id_escrita e filtrar toda busca de Animal/EventoSanitario/Doenca por essa fazenda antes de casar por número/nome.
- [CORRIGIDO nesta sessão] Corrigir exclusoes.py::_alvos para filtrar Servico/Parto/ControleLeiteiro/Sanidade/AgendaManual por fazenda_id, não só por numero_matriz, antes de incluir no cascata de exclusão.
- [CORRIGIDO nesta sessão] Corrigir POST /producao/bst/ajustar-proxima-aplicacao para exigir a fazenda do chamador e filtrar a leitura de Sanidade e a escrita de ParametroFazenda por ela.
- [CORRIGIDO nesta sessão] Corrigir alimentacao.py: encerrar_dieta, renomear/excluir produto de tabela nutricional, registrar_real — todas devem checar a posse do objeto pai antes de agir.
- [CORRIGIDO nesta sessão] Corrigir o webhook POST /cobranca/webhook/bb/{secret} para comparar o segredo contra um valor configurado (mesmo padrão de zapsign.py/asaas.py) antes de processar qualquer payload.
- [CORRIGIDO nesta sessão] Corrigir upload de documentos/fotos para sanitizar a extensão do arquivo (reaproveitar nome_seguro_storage(), já existente mas não chamado) e restringir o Content-Type salvo a uma allow-list.

P2 — Alto/Médio, próxima sprint

- [CORRIGIDO nesta sessão] Padronizar as ~20 ocorrências sistêmicas de rotas de criação usando get_fazenda_atual_id em vez de get_fazenda_id_escrita (RH, sanidade, farmácia, estoque, financeiro, animais, produção, reprodução) — risco baixo individualmente, mas repetitivo e fácil de varrer de uma vez com um teste arquitetural (grep) no CI.
- [CORRIGIDO nesta sessão] Corrigir as checagens de posse que toleram fazenda_id=None em PUT/DELETE de RH (rh_folha.py, rh_contratos.py, pessoas.py), sanidade e protocolos sanitários — trocar `fazenda\_id is not None and obj.fazenda\_id != fazenda\_id` por uma checagem que não abra mão da posse quando o token não tem fazenda selecionada.
- [CORRIGIDO nesta sessão] Corrigir o vazamento de EstoqueSemen em relatorios.py e nao_conformidades.py replicando o filtro já existente em relatorio_acasalamento.py.
- [CORRIGIDO nesta sessão] Restringir escrita no catálogo global de Touro (cadastro/genetica.py) a um papel mais alto que o módulo genérico 'parametros'.
- [CORRIGIDO nesta sessão] Escapar (html.escape ou template com autoescape) todo texto de usuário interpolado em corpo de e-mail: portal.py (corpo livre), financeiro.py (fornecedor_cliente no recibo), reproducao.py (diagnóstico/método), auth.py (nome no reset de senha).
- [CORRIGIDO nesta sessão] Trocar jinja2.Template() por jinja2.Environment(autoescape=select_autoescape(["html"])) em contrato_render.py, replicando o padrão já usado em contrato_equipe_render.py.
- [CORRIGIDO nesta sessão] Adicionar checagem de posse de fazenda em financeiro.py (vincular_evento_sanitario_reprodutivo, recibo por numero_lancamento) e em cartao_credito.py (conta_bancaria_id).
- [CORRIGIDO nesta sessão] Aplicar exigir_admin (ou equivalente) em chamados.py::atualizar_status_chamado.

- [CORRIGIDO nesta sessão] Fazer `cofre_acesso.py::encerrar_sessao` surtir efeito de fato — consultar `SessaoAcessoSuporte` no middleware de validação de token/modo suporte, não só gravar a flag.
- [CORRIGIDO nesta sessão] Decidir e aplicar: incluir `/cadastro/pessoas` nos prefixos bloqueados de modo suporte (ou um subconjunto de dados pessoais dele), dado que `salario_base/CPF/anexos pessoais` são tão sensíveis quanto o resto de RH já bloqueado.

P3 — Baixo/Informativo, quando houver tempo

- [CORRIGIDO nesta sessão] Remover o campo morto `secret_key` de `config.py` (ou dar a ele o mesmo guard de startup de `AUTH_SECRET`).
- [CORRIGIDO nesta sessão] Proteger ou remover `GET /telegram/status` (vaza a `allow-list` de chats sem autenticação).
- [CORRIGIDO nesta sessão] Validar esquema `http(s)://` em `link_rastreio` (`pedidos.py`) e `link_bula` (`farmacia.py`) no backend.
- [CORRIGIDO nesta sessão] Adicionar `maker-checker` (`aprovador` $\neq$ `solicitante`) em `cofre_acesso.py::aprovar_pedido`, mesmo que hoje nenhuma fazenda-piloto ative essa opção.
- [CONFIRMADO com o administrador da infraestrutura Railway nesta sessão] Revisar a heurística `_rodando_em_producao()` com quem administra a infraestrutura Railway, documentando explicitamente os formatos de `DATABASE_URL` cobertos. Confirmado: em produção, `DATABASE_URL` é a variável de referência do Railway `"${Postgres.DATABASE_URL}"`, que resolve para uma string começando com `"postgresql://"` — um dos dois prefixos já cobertos por `_rodando_em_producao()` (`fazenda/auth.py`). Nenhuma mudança de código necessária.
- [REQUER CONFIRMAÇÃO OPERACIONAL — não corrigível em código] Confirmar operacionalmente se `TELEGRAM_CHAT_FAZENDA` cobre 100% dos chats ativos, para fechar o vazamento residual de `LancamentoPendente` entre fazendas. Depende de dados de produção (lista real de chats configurados), não do código-fonte.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de cada issue.

--- ISSUE 1 ---

```
# [Segurança] Escalação de privilégio para dono via e-mail dono-equivalente – CORRIGIDO, falta
revisar/mergear

**Labels sugeridas:** `security`, `critical`

## Descrição do problema
`PUT /auth/preferencias` (backend/fazenda/api/routers/auth.py:518-527) só comparava o e-mail enviado
com `EMAIL_DONO`, ignorando o segundo e-mail dono-equivalente em `EMAILS_DONO_EQUIVALENTE`
(fazenda/auth.py:87-91, usado por `eh_email_dono_equivalente()`). Como essa rota só exige um login
ativo (qualquer papel, qualquer fazenda), qualquer usuário autenticado podia se autopromover a dono da
plataforma inteira enviando esse segundo e-mail uma única vez, sem nenhuma checagem de admin.

O mesmo bug existia em `POST/PUT /painel-cowdata/usuarios/{fazenda_id}/{usuario_id}`
(backend/fazenda/api/routers/painel_cowdata_usuarios.py:101-165), gated só por
`exigir_area_painel_cowdata("cadastros")` – um funcionário do CowData com acesso apenas a "cadastros"
podia criar/editar um login de qualquer fazenda-cliente com esse e-mail e senha à escolha.

## Evidência
```python
backend/fazenda/api/routers/auth.py:521 (ANTES)
if novo_email and novo_email.lower() == EMAIL_DONO: # só comparava com EMAIL_DONO
...
```python
# backend/fazenda/api/routers/painel_cowdata_usuarios.py:117,159 (ANTES)
novo = Usuario(..., email=(dados.email or "").strip() or None) # sem checagem nenhuma
u.email = dados.email.strip() or None # sem checagem nenhuma
...

## Impacto
Qualquer usuário autenticado (mesmo operador) podia obter acesso total de dono da plataforma – acesso
irrestrito a todos os tenants, Painel CowData completo, dados financeiros e de RH de todas as
fazendas-cliente.

## Correção sugerida (já aplicada nesta sessão)
- `auth.py`: trocar a condição para `eh_email_dono_equivalente(novo_email)`, e dentro dela recusar com
403 qualquer e-mail que não seja literalmente `EMAIL_DONO` (preservando o caminho de auto-atendimento
só para o dono original).
- `painel_cowdata_usuarios.py`: importar `eh_email_dono_equivalente` e recusar com 403 qualquer e-mail
dono-equivalente em ambas as rotas (criar e editar).
- Testes de regressão adicionados em `tests/test_auth_preferencias.py` e
`tests/test_painel_cowdata_usuarios.py`.

## Critérios de aceite
- [ ] Revisar o diff já aplicado em `auth.py` e `painel_cowdata_usuarios.py` nesta sessão.
- [ ] `pytest tests/test_auth_preferencias.py tests/test_painel_cowdata_usuarios.py
tests/test_auth_dono_equivalente.py` passa.
- [ ] Rodar a suíte completa de testes backend sem regressão.
- [ ] Confirmar que nenhuma outra rota grava `Usuario.email` sem essa checagem (grep por `email =` /
`email=` em modelos `Usuario`).

--- FIM ISSUE 1 ---
```

--- ISSUE 2 ---

```
# [Segurança] IDOR crítico: confirmação/desconfirmação de protocolos na Agenda sem checar fazenda

**Labels sugeridas:** `security`, `critical`

## Descrição do problema
`POST /agenda/realizados` e `DELETE /agenda/realizados/{evento_id}`
(backend/fazenda/api/routers/agenda.py) despacham para várias funções que buscam
`ProtocoloIatfAplicacao`, `ProtocoloInducaoLancamento`, `ProtocoloSanitarioAplicacao`,
`AplicacaoAgendada`, protocolo customizado e "lida" – nenhuma delas filtra por `fazenda_id`. Os
identificadores (`lancamento_id`, `aplicacao_id`, pares `data_prevista`/`dia`) são inteiros pequenos e
```

sequenciais ou datas previsíveis.

```
## Evidência
```python
agenda.py:2025-2031
aplicacoes = session.exec(
select(ProtocoloIatfAplicacao).where(
ProtocoloIatfAplicacao.data_prevista == data_prevista,
ProtocoloIatfAplicacao.dia == dia,
ProtocoloIatfAplicacao.realizada == False,
)
).all() # sem fazenda_id
```

Mesma forma em: agenda.py:1857-1966 (protocolo sanitário / aplicação agendada / vacina pré-parto),
rules/protocolo_customizado.py:127-133, rules/lida.py:106-112, e no lado DELETE (agenda.py:2439-2461,
2224-2241, 146-159, 141-160).

## Impacto
Qualquer usuário autenticado de qualquer fazenda-cliente pode confirmar OU reverter tratamento
sanitário/reprodutivo de outra fazenda, gravando `Sanidade` falsa referenciando o animal da vítima,
consumindo o próprio estoque do atacante, e apagando o registro de conformidade real da vítima.

## Correção sugerida
Reaproveitar o padrão já correto de `central_protocolos.py::_lançamento_ou_404`, que valida posse antes
de chamar as mesmas funções `marcar_realizado`/`desmarcar_realizado` – aplicar a mesma pré-validação
(ou passar `fazenda_id` para dentro dessas funções e filtrar todas as buscas por ele) em todos os
pontos de despacho de `agenda.py`.

## Critérios de aceite
- [ ] Toda busca dentro de `POST/DELETE /agenda/realizados` (IATF, indução, sanitário, customizado,
lida) filtra por `fazenda_id` do chamador antes de agir.
- [ ] Teste de regressão: usuário da fazenda B não consegue confirmar/desconfirmar protocolo/aplicação
pertencente à fazenda A via `POST`/`DELETE /agenda/realizados`.
- [ ] `GET /protocolo-iatf/concluidos` e `GET /protocolo-inducao-lactacao/concluidos`
(agenda.py:2464-2518) também passam a filtrar por fazenda_id (mesmo grupo de causa raiz).

--- FIM ISSUE 2 ---
```

--- ISSUE 3 ---

```
# [Segurança] Vazamento total de dados multi-tenant via exportação/e-mail do Portal

**Labels sugeridas:** `security`, `critical`

## Descrição do problema
`POST /portal/exportar` (via `executar_exportacao`, backend/fazenda/api/routers/portal.py:417-448)
monta um ZIP com 10 tabelas (`Animal`, `Servico`, `Parto`, `ControleLeiteiro`, `Estoque`,
`MovimentoEstoque`, `Sanidade`, `CompraAnimal`, `VendaAnimal`, `ContaGerencial`) sem NENHUM filtro de
`fazenda_id`. `POST /portal/email` com relatório (dre/rmca/custo_litro_leite) chama essas funções como
Python puro, sem passar `fazenda_id` – como o parâmetro é `Depends(get_fazenda_atual_id)`, a chamada
direta deixa esse argumento como o próprio objeto `Depends(...)` , que `fazenda_id_seguro()` converte
para `None`, desligando todo filtro interno.

## Evidência
```python
portal.py:421-427
query = select(cfg["model"])
if cfg["campo_data"] and item.get("data_inicio") and item.get("data_fim"):
...
linhas = [row.model_dump(mode="json") for row in session.exec(query).all()] # sem fazenda_id
```

```python
portal.py (envio de relatório por e-mail)
resultado = dre(data_inicio=item["data_inicio"], data_fim=item["data_fim"], centro_custo=None,
regime="competencia", session=session) # fazenda_id nunca passado
```

## Impacto
A rota exige só `user.papel == "admin"` (403 caso contrário) – ou seja, qualquer admin de QUALQUER
fazenda-cliente pode se enviar por e-mail um ZIP/CSV com os dados de todos os outros clientes da
plataforma: animais, produção, sanidade, compras/vendas e o livro financeiro completo.
```

```
## Correção sugerida
Passar `fazenda_id` (resolvido via `get_fazenda_id_escrita`/`get_fazenda_atual_id` no handler HTTP)
para `_executar_exportacao` e filtrar cada uma das 10 queries por ele; passar `fazenda_id`
explicitamente em toda chamada direta a `dre`/`rmca`/`custo_litro leite` dentro de `portal.py`.

## Critérios de aceite
- [ ] Toda tabela do `EXPORT_CATALOG` é filtrada por `fazenda_id` do solicitante.
- [ ] `dre`/`rmca`/`custo_litro leite` chamados a partir de `portal.py` sempre recebem `fazenda_id`
explícito.
- [ ] Teste de regressão: admin da fazenda B solicita exportação/e-mail e recebe só dados da fazenda B.

--- FIM ISSUE 3 ---
```

--- ISSUE 4 ---

```
# [Segurança] Importação de planilhas casa/cria registros globalmente por número, sem fazenda_id

**Labels sugeridas:** `security`, `critical`

## Descrição do problema
Quatro rotas de importação em `backend/fazenda/api/routers/importar.py` buscam
`Animal`/`EventoSanitario`/`Doenca` GLOBALMENTE por `numero`/`nome` (sem filtro de fazenda) e, quando
ausentes, criam esses registros com `fazenda_id=None`:
- `POST /importar/animais_cadastro` (487-544) – **crítica**
- `POST /importar/animais_genealogia` (547-595) – alta
- `POST /importar/dairycomp` (658-729) – **crítica**
- `POST /importar/calendario_sanitario` (732-813) – alta

Nenhuma das quatro tem dependência de `fazenda_id`/usuário autenticado no próprio handler (a
autenticação vem só do gate de módulo no include do router).

## Evidência
```python
importar.py:490 (animais_cadastro) e 664 (dairycomp) – mesmo padrão
animal = session.exec(select(Animal).where(Animal.numero == numero)).first()
if not animal:
 animal = Animal(numero=numero, ativo=True) # sem fazenda_id
```

## Impacto
Números de animal (e nomes de evento sanitário) são pequenos e comumente reutilizados entre fazendas
diferentes. Uma fazenda importando seu próprio rebanho/calendário pode sobrescrever silenciosamente o
cadastro de animal, genealogia, data de nascimento ou parto de OUTRA fazenda, ou anexar dados de
calendário sanitário à configuração global de outro tenant.

## Correção sugerida
Resolver `fazenda_id` via `get_fazenda_id_escrita` em cada uma das quatro rotas e filtrar toda busca
por `numero`/`nome` por essa fazenda antes de decidir entre atualizar ou criar.
```

```
## Critérios de aceite
- [ ] As 4 rotas resolvem e filtram por `fazenda_id` do chamador.
- [ ] Teste de regressão: importar um CSV com número de animal que já existe em OUTRA fazenda não altera
o registro da outra fazenda, cria um novo registro na fazenda do importador.

--- FIM ISSUE 4 ---
```

--- ISSUE 5 ---

```
# [Segurança] Exclusão em cascata de animal pode apagar histórico de outra fazenda

**Labels sugeridas:** `security`, `critical`

## Descrição do problema
Em `backend/fazenda/api/routers/exclusoes.py:502-607` (`_alvos`, usado por `POST /exclusoes/confirmar`
e `pendentes/{id}/aprovar`), a busca do `Animal` é corretamente escopada por fazenda, mas as buscas
seguintes de `Servico`, `Parto`, `ControleLeiteiro`, `Sanidade` (e `AgendaManual` para retenção de
placenta) filtram só por `numero_matriz`, SEM `fazenda_id` – e esses objetos são de fato apagados por
`excluir_alvos_em_ordem`.

## Evidência
```python
```

```
exclusoes.py
servicos = session.exec(select(Servico).where(Servico.numero_matriz == id_)).all() # sem fazenda_id
partos = session.exec(select(Parto).where(Parto.numero_matriz == id_)).all() # sem fazenda_id
...
return impacto, [animal, *servicos, *partos, *controles, *sanidades, ...] # depois deletados
```



## Impacto



Números de animal são pequenos e reutilizados entre fazendas diferentes (confirmado em comentário do próprio `upload.py`: "a vaca '18' existe em várias [fazendas]"). Um admin excluindo seu próprio animal "464" também apaga o histórico reprodutivo/produtivo/sanitário do animal "464" de OUTRA fazenda, se os números coincidirem – bug disparável em uso normal, não só por má-fé.



## Correção sugerida



Filtrar `Servico`/`Parto`/`ControleLeiteiro`/`Sanidade`/`AgendaManual` por `fazenda_id` (a mesma fazenda já resolvida para o `Animal`) antes de incluí-los na lista de alvos a excluir.



## Critérios de aceite



- [ ] Todas as sub-buscas de `_alvos` (tipo "animal" e "parto") filtram por `fazenda_id`.
- [ ] Teste de regressão: excluir o animal "X" da fazenda A não afeta nenhum registro do animal "X" da fazenda B.



--- FIM ISSUE 5 ---


```

--- ISSUE 6 ---

```
# [Segurança] Alimentação: encerrar dieta e ações relacionadas sem checar fazenda

**Labels sugeridas:** `security`, `critical`

## Descrição do problema
Em `backend/fazenda/api/routers/alimentacao.py`:
- `PUT /alimentacao/dietas/{id}/encerrar` (1809-1819) – **crítica**: nenhuma dependência de fazenda; qualquer usuário pode encerrar a dieta ativa de outro tenant.
- `POST /alimentacao/dietas/{id}/real` (1827-1844) – alta: `dieta_id` referenciado nunca é checado contra a fazenda do chamador.
- `PUT`/`DELETE /alimentacao/tabela-nutricional/produtos/{id}` (1261-1287) – alta: sem fazenda_id, ao contrário de todas as rotas irmãs do bloco.
- `GET /alimentacao/dietas/{id}/comparativo` e `apresentacao_dieta` (1761-1879) – média: o cabeçalho da dieta vaza entre fazendas mesmo com os subitens corretamente filtrados.

## Evidência
```python
alimentacao.py:1809-1819
dieta = session.get(DietaLancamento, dieta_id)
if not dieta:
 raise HTTPException(404, ...)
dieta.data_efetivo_encerramento = dados.data_efetivo_encerramento # sem checar fazenda_id
```

## Impacto
Interrupção da baixa automática de estoque e da apuração de outra fazenda; poluição do comparativo programado-vs-real de outro tenant; renomeação/exclusão do catálogo nutricional de outro tenant.

## Correção sugerida
Adicionar `fazenda_id: int = Depends(get_fazenda_id_escrita)` a todas as rotas listadas e checar `dieta.fazenda_id == fazenda_id` (ou equivalente para `TabelaNutricionalProduto`) antes de qualquer leitura/escrita.

## Critérios de aceite


- [ ] As 4 rotas listadas checam posse antes de agir.
- [ ] Teste de regressão cobrindo cada uma.



--- FIM ISSUE 6 ---


```

--- ISSUE 7 ---

```
# [Segurança] Webhook do Banco do Brasil (cobrança) não valida o segredo do path

**Labels sugeridas:** `security`, `high`

## Descrição do problema
```

`POST /cobranca/webhook/bb/{secret}` (backend/fazenda/api/routers/cobranca.py:102-131) captura `secret` do path mas nunca o compara a nenhum valor configurado – só checa se `BB\_CLIENT\_ID` está definido. O `status` do payload é confiado diretamente, sem reconfirmação via API do Banco do Brasil.

Evidência

```
```python
async def cobranca_webhook_bb(secret: str, request: Request, session=...) -> dict:
if not settings.bb_client_id:
raise HTTPException(403, "Banco do Brasil não configurado")
payload = await request.json()
if "txid" in payload:
pix = session.exec(select(CobrancaPix).where(CobrancaPix.txid == payload["txid"])).first()
if pix and payload.get("status") in ("CONCLUIDA", "paga"):
pix.status = "paga" # secret nunca comparado, status nunca reconfirmado
```
```

Contraste: `zapsign.py:25-28` e `asaas.py:143-185` comparam o segredo e reconfirmam o status via API autenticada antes de mudar qualquer estado.

Impacto

Qualquer um que descubra/adivinhe essa URL (caminho previsível, sem segredo real) pode marcar boletos/Pix como pagos, sem nunca ter havido pagamento real.

Correção sugerida

Comparar `secret` contra um valor configurado (`settings.bb\_webhook\_secret`, a criar se não existir) com `hmac.compare\_digest`, e reconfirmar o status via chamada autenticada à API do BB antes de gravar `pix.status = "paga"`, no mesmo padrão de `asaas.py`.

Critérios de aceite

- [] Webhook rejeita requisições sem o segredo correto.
- [] Status só é gravado após reconfirmação via API do BB.
- [] Teste de regressão cobrindo os dois casos.

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Segurança] Upload de documentos/fotos: extensão de arquivo não sanitizada (path traversal no Storage) + Content-Type livre

****Labels sugeridas:**** `security`, `high`

Descrição do problema

Em `backend/fazenda/api/routers/documentos.py:68-97` e `fotos.py:102-132`, a extensão do arquivo é derivada de tudo após o último`.` do nome enviado pelo cliente, sem rejeitar `/` ou `..`, e concatenada direto na URL REST do Supabase Storage (`fazenda/rules/supabase\_storage.py:74-75`) sem sanitização – apesar de já existir `nome\_seguro\_storage()` no mesmo módulo, nunca chamado por esses dois uploads. O mesmo caminho deixa o cliente definir o `Content-Type` salvo, servido depois com `Content-Disposition: inline`.

Evidência

```
```python
nome_original = file.filename or "arquivo"
extensao = f".{nome_original.rsplit('.', 1)[-1].lower()}" if "." in nome_original else ""
...
caminho = f"{prefixo_pasta}/{prefixo_nome}{seq:04d}{extensao}" # sem sanitização
enviar_arquivo(caminho, conteudo, ...)
```
```

Impacto

Um nome de arquivo como `evil.png/../../../../outra-fazenda/arquivo.pdf` pode escrever/sobrescrever fora do prefixo `fazenda-{id}/...` esperado, potencialmente em outro tenant, dependendo de como o gateway do Supabase Storage trata `..` no path. Um `Content-Type` como `text/html`/`image/svg+xml` servido inline é risco de XSS armazenado se essa rota de download for aberta na mesma origem autenticada.

Correção sugerida

Chamar `nome\_seguro\_storage()` (já existente) em ambos os pontos de upload antes de montar o caminho; restringir `Content-Type` salvo a uma allow-list de tipos esperados (PDF, imagens comuns).

Critérios de aceite

- [] `documentos.py` e `fotos.py` sanitizam a extensão via `nome\_seguro\_storage()`.

- [] `Content-Type` é validado contra allow-list antes de salvar.
- [] Teste de regressão com nome de arquivo contendo `../`.

--- FIM ISSUE 8 ---

--- ISSUE 9 ---

[Segurança] Padronizar rotas de escrita para usar `get_fazenda_id_escrita` (padrão sistêmico, ~20 arquivos)

Labels sugeridas: `security`, `medium`, `tech-debt`

Descrição do problema

Dezenas de rotas de CRIAÇÃO usam `Depends(get_fazenda_atual_id)`` (dependência de leitura, que tolera `fazenda_id=None`` para tokens legados/ambíguos) em vez de `Depends(get_fazenda_id_escrita)`` (que recusa com 409 em vez de gravar `fazenda_id=None``). No caso-limite documentado (token sem claim `"fid"`, ex.: usuário vinculado a 2+ fazendas que ainda não selecionou uma), isso cria registros órfãos silenciosamente em vez de recusar a escrita – a mesma classe de bug que o próprio código já documenta como causa raiz de um incidente anterior ("D6 sumindo da Agenda").

Evidência (amostra representativa das ~20 ocorrências)

- `cadastro/animais.py``: `criar_animal`, `criar_raca`, `criar_grau_sangue`
- `producao.py``: `criar_pesagens`, `criar_qualidade_leite`, `criar_entrega_leite`
- `reproducao.py``: `adicionar_animais_iatf`
- `financeiro.py``: `criar_conta_corrente`, `criar_centro_custo`, `criar_conta_gerencia`, `crud_nome_ativo_financeiro``
- `cartao_credito.py``: `criar_cartao`
- `cadastro/servicos.py``: `criar_metodo_servico`, `atualizar_metodo_servico`
- `sanidade.py``: `criar_calendario`, `criar_cronograma_manual`, `registrar_colostragem`
- `cadastro/sanitario.py``: `POST /agendamentos-pesagem`, `/eventos-sanitarios`, `/exames`
- `cadastro/lida.py``, `cadastro/protocolos_customizados.py``: `criar_lida`, `criar_protocolo_customizado`
- `compra_semen.py``, `compra_animal.py``, `venda_animal.py``
- `baixas.py``: `marcar_a_descartar`
- `movimentacoes.py``: `criar_motivo`, `atualizar_motivo`, `salvar_parametro_agendamento`
- `lotes.py``: `criar_lote`, `atualizar_lote`
- `safras.py``: `criar_safras`, `atualizar_safras`
- `cadastro/genetica.py``: `criar_estoque_semen`, `atualizar_estoque_semen`
- `alertas_indicador.py``: `criar_alerta` (o mais sensível: um alerta com `fazenda_id=NULL` é avaliado sem filtro, agregando dados de todos os tenants no "valor_atual" mostrado a um único usuário)

Impacto

Individualmente baixo/médio (requer o caso-limite de token sem fazenda selecionada), mas repetitivo – cada ocorrência é um ponto potencial de dado órfão ou vazamento cross-tenant no caso-limite.

Correção sugerida

Trocar `Depends(get_fazenda_atual_id)`` por `Depends(get_fazenda_id_escrita)`` em cada rota de criação listada. Considerar um teste arquitetural no CI (grep/AST) que falhe se uma rota `POST`` nova instanciar um modelo com `fazenda_id=`` vindo de uma variável resolvida por `get_fazenda_atual_id``.

Critérios de aceite

- [] Todas as ocorrências listadas trocadas para `get_fazenda_id_escrita``.
- [] Teste arquitetural adicionado ao CI para prevenir recorrência.

--- FIM ISSUE 9 ---

--- ISSUE 10 ---

[Segurança] IDOR em RH: folha, contratos e pessoas aceitam edição/exclusão cross-tenant com token sem fazenda selecionada

Labels sugeridas: `security`, `high`

Descrição do problema

Em `rh_folha.py`` (17 ocorrências), `rh_contratos.py`` (11 ocorrências) e `pessoas.py`` (5 ocorrências), as rotas PUT/DELETE/GET-de-anexo usam a checagem `if not obj or (fazenda_id is not None and obj.fazenda_id != fazenda_id): 404``. Quando `fazenda_id`` é `None`` – estado normal e válido de um token de usuário vinculado a 2+ fazendas que ainda não chamou `/auth/selecionar-fazenda`` – a checagem inteira vira `if not obj``, ou seja, a posse não é verificada.

Evidência

```python

# rh\_folha.py:535-543 (representativo das 17 ocorrências)

```

registro = session.get(FolhaPagamento, registro_id)
if not registro or (fazenda_id is not None and registro.fazenda_id != fazenda_id):
 raise HTTPException(404, "Registro de folha não encontrado")
...

Rotas afetadas incluem folha de pagamento, guias, férias, 13º, rescisões, vales e comprovantes
(rh_folha.py); empreitadas, contratos, diárias, vale-avulso (rh_contratos.py); e edição/exclusão de
Pessoa + download/exclusão de anexos RG/CPF/holerite (pessoas.py).

Impacto
Um portador de token válido (mas sem fazenda selecionada) pode ler, editar ou excluir folha de
pagamento, férias, rescisão, vale, contrato de terceiro, ou baixar/excluir documentos pessoais
(RG/CPF/holerite) de QUALQUER fazenda-cliente, só incrementando o id.

Correção sugerida
Trocar a checagem tolerante por uma que não abra mão da posse quando `fazenda_id is None` nessas rotas
de mutação/download sensível – ex.: usar `get_fazenda_id_escrita` também para leitura de objeto
existente nessas rotas específicas (não só criação), já que RH é dado suficientemente sensível para não
tolerar o caso-limite.

Critérios de aceite
- [] As 33 ocorrências (17+11+5) corrigidas para não tolerar `fazenda_id is None` na checagem de
posse.
- [] Teste de regressão simulando token sem `fid` contra objeto de outra fazenda.
--- FIM ISSUE 10 ---

```

--- ISSUE 11 ---

```

[Segurança] Catálogo global de touros (genética) editável/excluível por qualquer funcionário

Labels sugeridas: `security`, `high`

Descrição do problema
`Touro` é deliberadamente um catálogo global (sem `fazenda_id`) usado por todas as fazendas-cliente. As
rotas de escrita em `backend/fazenda/api/routers/cadastro/genetica.py:403-509` (`POST/PUT/DELETE
/touros...`, `POST /touros/recarregar-catalogo`) não têm nenhum gate de papel além de
`exigir_modulo("parametros")` – permissão comum de funcionário, não admin/dono.

Evidência
```python
@router.delete("/touros/{touro_id}")
def excluir_touro(touro_id: int, session=...) -> dict:
    t = session.get(Touro, touro_id)
    if not t:
        raise HTTPException(404, "Touro não encontrado")
    session.delete(t) # sem checagem de papel além de exigir_modulo("parametros")
...

## Impacto
Qualquer funcionário comum de qualquer fazenda-cliente com o módulo "parametros" pode reescrever,
excluir ou reimportar em massa o catálogo de touros usado por TODOS os outros clientes da plataforma
(usado em ordenação TPI/NM$, sugestões de acasalamento etc.), sem forma de escopar ou desfazer por
tenant.

## Correção sugerida
Restringir as 4 rotas a um papel mais alto – no mínimo `exigir_admin`, idealmente equivalente a
`exigir_dono`/CowData, dado que afeta todos os tenants.

## Critérios de aceite
- [ ] As 4 rotas exigem papel elevado.
- [ ] Teste de regressão: funcionário comum recebe 403 ao tentar excluir/editar Touro.
--- FIM ISSUE 11 ---

```

--- ISSUE 12 ---

```

# [Segurança] Vazamento de estoque de sêmen entre fazendas em relatórios

**Labels sugeridas:** `security`, `high`

## Descrição do problema
`GET /relatorios/manejo` (backend/fazenda/api/routers/relatorios.py:75-82) e `GET /nao-conformidades/`

```

(`nao_conformidades.py:61`) fazem ``select(EstoqueSemen)`` sem filtro de ``fazenda_id``, expondo ``touro_nome`/`codigo`/`central`/`tipo`/`doses`` de estoque de sêmen de TODAS as fazendas a qualquer usuário autenticado.

Evidência

```
```python
```

```
semen = [s.model_dump() for s in session.exec(select(EstoqueSemen)).all()] # sem fazenda_id
```
```

Impacto

É exatamente a classe de vazamento que ``relatorio_acasalamento.py:25-30`` já corrigiu e documenta explicitamente como "informação comercial de genética dos concorrentes" – a correção não foi replicada nesses dois outros pontos.

Correção sugerida

Replicar o filtro de ``relatorio_acasalamento.py`` (`fazenda_id` do chamador) nas duas consultas.

Critérios de aceite

- [] As duas rotas filtram ``EstoqueSemen`` por ``fazenda_id``.
- [] Teste de regressão citando o mesmo padrão de ``tests/test_isolamento_relatorios_fornecedor.py``.

--- FIM ISSUE 12 ---

--- ISSUE 13 ---

[Segurança] IDOR em Financeiro: vincular evento sanitário/reprodutivo e recibo por número de lançamento

****Labels sugeridas:**** ``security``, ``high``

Descrição do problema

- ``POST /financeiro/vincular-evento-sanitario-reprodutivo`` (`financeiro.py:1684-1699`): sem ``fazenda_id`` algum; qualquer usuário pode vincular ``Sanidade`/`ExameResultado`/`Servico`` de outro tenant a um ``numero_lancamento`` à escolha.
- ``GET .../destinatario-recibo`` e ``POST .../recibo/enviar`` (`financeiro.py:4158-4207`): ``numero_lancamento`` é sequencial e global (sem filtro por fazenda em ``_proximo_numero_lancamento``) e as duas rotas não checam fazenda – vazam nome/e-mail do fornecedor de outro tenant e permitem enviar e-mail (com dados de outro tenant) para destinatário arbitrário.
- ``POST/PUT /financeiro/cartoes`` (`cartao_credito.py:118-150`): aceita ``conta_bancaria_id`` de outra fazenda sem checar posse, expondo depois dados bancários dela em ``pagar_fatura_cartao``.

Evidência

```
```python
```

```
obj = session.get(modelo, item_id)
```

```
if obj:
```

```
obj.numero_lancamento_vinculado = dados.numero_lancamento # sem checar fazenda_id
```

```
```
```

Impacto

Vazamento de dados financeiros/bancários entre tenants e uso do remetente de e-mail da plataforma para enviar dados de terceiros a um endereço arbitrário.

Correção sugerida

Adicionar ``fazenda_id: int = Depends(get_fazenda_id_escrita)`` (ou leitura segura) e checar posse do objeto antes de agir em cada uma das três rotas.

Critérios de aceite

- [] As três rotas checam posse antes de ler/escrever.
- [] Teste de regressão para cada uma.

--- FIM ISSUE 13 ---

--- ISSUE 14 ---

[Segurança] Injeção de HTML não escapado em e-mails e no gerador de contrato

****Labels sugeridas:**** ``security``, ``medium``

Descrição do problema

Cinco pontos interpolam texto de usuário em HTML sem escapar:

1. ``contrato_render.py:14,73`` usa ``jinja2.Template()`` (autoescape OFF por padrão) para renderizar ``contrato_cowdata.html``, interpolando ``fazenda_nome`/`endereco`/`representante_nome`/`representante_cpf`` (cadastráveis, e sobrescrevíveis via

query string em `GET /fazendas/{id}/contrato/modelo`) – servido com `Content-Disposition: inline`. O irmão `contrato\_equipe\_render.py` já usa `Environment(autoescape=select\_autoescape(["html"]))` corretamente.

2. `portal.py:286-289` – corpo livre do e-mail do Portal.
3. `financeiro.py:4197-4200` – `fornecedor\_cliente` no e-mail de recibo.
4. `reproducao.py:720-729` – `diagnostico`/`metodo\_diagnostico` (sem validação por enum em `PUT /servicos/{id}`) no e-mail de diagnóstico.
5. `auth.py:330-335` – `user.nome` no e-mail de redefinição de senha (impacto baixo, majoritariamente self-XSS).

Evidência

```
```python
```

```
contrato_render.py
```

```
from jinja2 import Template
```

```
return Template(arquivo.read_text(encoding="utf-8")).render(**contexto) # autoescape OFF
```

```
portal.py
```

```
corpo_html = f"<p>{dados.corpo}</p>" # sem escapar
```

```
financeiro.py
```

```
corpo_html = f"<p>...({conta.fornecedor_cliente or '-'}, ...)</p>" # sem escapar
```

```
```
```

Impacto

HTML/links/imagens podem ser injetados em e-mails enviados pela plataforma a usuários reais (phishing, tracking pixels), e o contrato gerado pode ter sua integridade comprometida por HTML/script injetado nos dados cadastrais da fazenda.

Correção sugerida

- `contrato\_render.py`: trocar para `Environment(autoescape=select\_autoescape(["html"]))`, igual ao irmão.

- Demais 4 pontos: aplicar `html.escape()` (ou equivalente) ao texto de usuário antes de interpolar no HTML do e-mail.

Critérios de aceite

- [] `contrato\_render.py` usa autoescape.
- [] Os 4 pontos de e-mail escapam o texto de usuário.
- [] Teste de regressão confirmando que `<script>`/`<img onerror>` chega escapado ao corpo final.

--- FIM ISSUE 14 ---

--- ISSUE 15 ---

[Segurança] Painel CowData / Cofre de Acesso: revogação de sessão de suporte não é aplicada no middleware

****Labels sugeridas:**** `security`, `high`

Descrição do problema

`POST /painel-cowdata/cofre/sessoes/{id}/encerrar` (cofre_acesso.py:253-266) só grava `sessao.encerrada\_em` no banco e uma linha de auditoria. O token de suporte assinado (emitido em `solicitar\_acesso`/`aprovar\_pedido`) é validado de forma totalmente stateless – `\_validar\_token\_payload`, `get\_current\_user` e `\_bloquear\_modosuporte` (main.py) nunca consultam `SessaoAcessoSuporte` para saber se a sessão foi encerrada.

Evidência

grep confirma que `SessaoAcessoSuporte` só é referenciado dentro de `cofre\_acesso.py` – nunca no caminho de auth/middleware.

Impacto

Um token de suporte continua com acesso total (exceto os prefixos já bloqueados) por até 30 minutos mesmo após ser "encerrado" manualmente – contradizendo o próprio comentário do modelo ("pode ser encerrada manualmente antes"). Um token de suporte vazado ou revogado por suspeita de uso indevido não é efetivamente cortado.

Correção sugerida

No caminho de validação de token (ou em `\_bloquear\_modosuporte`), quando o payload carregar `sessao\_suporte\_id`, consultar `SessaoAcessoSuporte` e rejeitar a requisição se `encerrada\_em` is not None`.

Critérios de aceite

```
- [ ] Uma sessão de suporte encerrada manualmente deixa de autorizar requisições imediatamente (não só após expirar).
- [ ] Teste de regressão: encerrar sessão e confirmar que o token correspondente passa a ser rejeitado.
--- FIM ISSUE 15 ---
```

```
--- ISSUE 16 ---
```

```
# [Segurança] Itens diversos de baixa severidade / informativos
```

```
**Labels sugeridas:** `security`, `low`
```

```
## Descrição do problema (agrupado – cada item é pequeno e independente)
```

1. `**config.py:9**` – ``secret_key: str = "change-me"`` é campo morto (nenhum código lê ``settings.secret_key``), mas tem aparência de segredo esquecido.
2. `**telegram.py:688-696**` – ``GET /telegram/status`` é público (sem autenticação) e vaza a allow-list completa de ``TELEGRAM_ALLOWED_CHAT_IDS``.
3. `**telegram.py:100-116 + aprovacoes.py:60-77**` – um chat do Telegram autorizado mas ausente de ``TELEGRAM_CHAT_FAZENDA`` gera ``LancamentoPendente`` com ``fazenda_id=None``, visível/acionável por todas as fazendas.
4. `**cofre_acesso.py:206-234**` – ``aprovar_pedido``/``negar_pedido`` não impedem que o mesmo usuário que solicitou acesso de suporte também aprove seu próprio pedido (sem impacto real hoje porque nenhuma fazenda-piloto ativa essa opção).
5. `**chamados.py:70-88**` – ``PUT /chamados/{id}`` não exige papel elevado para fechar um ticket e gravar uma "resposta" que deveria representar o suporte oficial.
6. `**pedidos.py (link_rastreio) / farmacia.py (link_bula)**` – campos de URL livre sem allow-list de esquema (``javascript:`` teoricamente armazenável).
7. `**indicadores.py:313**` – ``select(Lote)`` sem filtro de fazenda, ao contrário das consultas irmãs no mesmo handler.
8. `**baixas.py:73-129**` – ``marcar_a_descartar`` busca ``Animal`` sem filtro de fazenda quando o token não tem fazenda selecionada.

```
## Impacto
```

```
Individualmente baixo (requer configuração incomum, ou vaza só metadado não sensível) – agrupados aqui para não gerar spam de issues triviais.
```

```
## Correção sugerida
```

- Remover ``secret_key`` morto (ou dar o mesmo guard de ``AUTH_SECRET``).
- Proteger ``GET /telegram/status`` com autenticação ou removê-lo.
- Confirmar cobertura de ``TELEGRAM_CHAT_FAZENDA`` operacionalmente.
- Adicionar checagem "aprovador ≠ solicitante" em ``cofre_acesso.py``.
- Exigir ``exigir_admin`` em ``PUT /chamados/{id}``.
- Validar esquema ``http(s)://`` em ``link_rastreio``/``link_bula``.
- Filtrar ``Lote`` por fazenda em ``indicadores.py``.
- Usar ``get_fazenda_id_escrita`` (ou checagem de posse não tolerante) em ``marcar_a_descartar``.

```
## Critérios de aceite
```

- [] Cada um dos 8 itens tratado e coberto por pelo menos um teste onde aplicável.

```
--- FIM ISSUE 16 ---
```